

信息学奥赛 CSP 满分之路

CSP-J/S第一轮 2024
原创全真模拟试卷集

王津 陈真 刘俊星 夏宗强 祝敏磊 编著

人民邮电出版社

北京

图书在版编目 (C I P) 数据

信息学奥赛CSP满分之路 : CSP-J/S第一轮原创全真
模拟试卷集. 2024 / 王津等编著. -- 北京 : 人民邮电
出版社, 2024. 6

(Coding Kids)

ISBN 978-7-115-64379-7

I. ①信… II. ①王… III. ①程序设计—习题集
IV. ①TP311.1-44

中国国家版本馆CIP数据核字(2024)第093839号

内 容 提 要

信息学奥赛初赛 (CSP 第一轮) 是信息学奥林匹克竞赛 (以下简称 “信奥赛”) 的起始阶段, 也是参赛者迈向更高层次竞赛的必经之路。本题集是信奥赛初赛的重要学习和训练资料, 内容涵盖了信奥赛初赛涉及的计算机科学基础知识、编程技能的实践应用以及算法设计与分析等多项内容。题目设计注重实际应用和思维拓展, 难度适中, 既有对基础知识的考查, 也有一定难度的挑战性题目, 适用于对计算机科学和编程感兴趣的青少年。无论是初学者还是有一定基础的选手, 都可以从本题集中找到适合自己的学习内容和练习题目。此外, 本题集还可以作为学校信息学竞赛教学的辅助教材, 帮助教师制订合适的教学计划和练习方案, 引导学生系统地学习和掌握计算机科学的基础知识和技能, 更好地备战信奥赛初赛。

-
- ◆ 编 著 王 津 陈 真 刘俊星 夏宗强 祝敏磊
责任编辑 魏勇俊
责任印制 胡 南
 - ◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路11号
邮编 100164 电子邮件 315@ptpress.com.cn
网址 <https://www.ptpress.com.cn>
三河市中晟雅豪印务有限公司印刷
 - ◆ 开本: 787×1092 1/16
印张: 15.25 2024年6月第1版
字数: 316千字 2024年6月河北第1次印刷
-

定价: 59.80元

读者服务热线: (010)84084456-6009 印装质量热线: (010)81055316

反盗版热线: (010)81055315

广告经营许可证: 京东市监广登字 20170147 号

编 委 会

主 任

周祖松 重庆市育才中学校

王孝东 南京市金陵中学河西分校

委 员

陈 睿 重庆市第一中学校

陈俊先 成都外国语学校

陈旭龙 广州市铁一中学

代 文 上海外国语大学附属杭州学校

姜祥瑜 东莞市东莞中学松山湖学校

李欣泽 香港浸会大学（珠海）

刘卫华 湖北省武昌实验中学

刘志升 哈尔滨市第三中学校

钱 涛 上海市上海中学

吴金全 武汉市武钢三中

向 欣 四川省广元外国语学校

杨慧蓉 乌鲁木齐市第一中学

杨 平 江西省广丰贞白中学

姚肖华 汕头市金山中学

张金苗 合肥市第四十五中学

张新华 广州市第六中学

郑凌云 四川省绵阳南山中学

邹 毅 衡阳市第八中学

（排名不分先后，仅按姓氏音序排列）

前言

CSP-J/S 初赛介绍

CSP-J/S 初赛，是中国计算机学会（CCF）非专业级软件能力认证（Certified Software Professional Junior/Senior）初赛的简称，是由 CCF 统一组织的评价计算机非专业人士算法和编程能力的活动。

CSP-J/S 初赛是信息学奥林匹克竞赛选拔优秀选手进入更高层次竞赛的重要环节，旨在考查参赛者在计算机科学和编程方面的基础知识和能力。初赛题型通常包括单项选择题、阅读程序题和完善程序题，涉及的知识点包括但不限于计算机基础知识、算法、数据结构、编程语言等。在初赛中，参赛者需要展现出扎实的编程基础和良好的逻辑思维能力，能够灵活运用所学知识解决实际问题。

由于近些年广大青少年学习编程的热情和信息学奥林匹克竞赛参与程度的提高，报名参加参赛的人数逐年增加，各省市地区 CSP-J/S 初赛的通过率连年降低，造成很多有希望获奖的选手折戟初赛，白白浪费了一年的努力。因此，需要广大参赛者认真对待初赛，积极准备，进而充分展示自己的实力，顺利进入复赛。

本题集的特色和用法

在信息科技日新月异的今天，我们深知青少年是国家的未来，而编程与计算思维是他们必备的技能之一。这本 CSP-J/S 初赛题集正是为了激发青少年对编程的兴趣，引导和培养他们的创新思维和解决问题的能力而精心策划和编写的。

本题集在整理和总结往年初赛真题的基础上，为 CSP-J 和 CSP-S 两个组别，各推出 10 套原创题目，涵盖了 CSP-J/S 初赛所涉及的各种编程知识点。从基础语法到算法逻辑，从数据结构到程序设计思想，既有基础的编程题目，也有具有一定挑战性的思考题。我们相信，无论你是初学者，还是有一定编程基础的选手，通过本题集的练习和挑战，一定能够在 CSP-J/S 初赛中取得优异的成绩，顺利进入复赛。同时，我们希望本题集能够成为指导老师的得力助手。在题目设计上，我们注重逻辑性和教育性，旨在帮助指导老师更好地了解学生的学习状况，为他们提供更有针对性的指导和训练。

本题集旨在帮助参赛者深入理解信息学奥林匹克竞赛的考试要求和内容，提高编程和解决问题的能力，并为参赛者提供一个系统地复习和练习的平台。考虑到读者学习程度和精力上的差异，本题集除了参考答案，还提供了较为细致的题目解析。同时建议读者熟悉和模拟真实考试环境，提高自己的应试能力。面对模拟考试成绩，认真总结自己的表现和不足，及时调整学习策略。

答案解析

为节省篇幅，本题集答案和解析将以在线模式分享给各位读者。读者既可扫码下载每套题的答案及解析，也可通过图灵社区本书页面（ituring.com.cn/book/3329）右侧的“随书下载”下载。

学术交流

读者可以扫描下方二维码进入本书读者交流群，了解初赛的更多信息，并与全国的信息学奥林匹克竞赛选手交流互动，共同进步。



致谢

首先，我们衷心感谢所有参与编写和审阅本题集的专家、教练和工作人员。正是因为你们的辛勤付出和无私奉献，本题集才得以顺利出版，为广大参赛者提供了宝贵的复习和练习资料。

其次，我们要感谢那些在教学一线默默耕耘的指导老师。是你们用自己的知识和经验，引领更多的孩子走进编程的世界，激发他们对计算机科学和编程的兴趣。你们的不断付出和坚持，为培养我国的计算机编程人才奠定了坚实的基础。

最后，我们要感谢所有选择使用本题集的参赛者。你们的信任和支持，是我们编写题集的最大动力。希望你们能够通过本题集，加深对计算机科学和编程的理解，提高自己的编程能力和解决问题的能力，在信息学奥林匹克竞赛中取得优异的成绩。

最后的话

由于信息学奥林匹克竞赛的内容和要求会不断更新和变化，我们也将不定期更新题集和相关资料，确保读者的学习内容与考试要求保持同步。也希望广大读者针对本题集提供更多宝贵意见，以便修订再版时进行改进。

希望本题集能够成为广大青少年编程学习旅程中的一盏明灯，照亮前行的道路。

祝所有青少年参赛者在 CSP-J/S 初赛取得优异的成绩！

《信息学奥赛 CSP 满分之路》编委会

2024 年 2 月

目 录

普及组 CSP-J 2024 初赛模拟卷 1	1
普及组 CSP-J 2024 初赛模拟卷 2	11
普及组 CSP-J 2024 初赛模拟卷 3	21
普及组 CSP-J 2024 初赛模拟卷 4	31
普及组 CSP-J 2024 初赛模拟卷 5	41
普及组 CSP-J 2024 初赛模拟卷 6	51
普及组 CSP-J 2024 初赛模拟卷 7	64
普及组 CSP-J 2024 初赛模拟卷 8	77
普及组 CSP-J 2024 初赛模拟卷 9	89
普及组 CSP-J 2024 初赛模拟卷 10	101
提高组 CSP-S 2024 初赛模拟卷 1	112
提高组 CSP-S 2024 初赛模拟卷 2	124
提高组 CSP-S 2024 初赛模拟卷 3	137
提高组 CSP-S 2024 初赛模拟卷 4	149
提高组 CSP-S 2024 初赛模拟卷 5	160
提高组 CSP-S 2024 初赛模拟卷 6	173
提高组 CSP-S 2024 初赛模拟卷 7	187
提高组 CSP-S 2024 初赛模拟卷 8	199
提高组 CSP-S 2024 初赛模拟卷 9	209
提高组 CSP-S 2024 初赛模拟卷 10	223

普及组 CSP-J 2024 初赛模拟卷 1

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在标准 ASCII 码表中，已知英文字母 Z 的 ASCII 码十进制表示是 90，那么英文字母 B 的 ASCII 码二进制表示是（ ）。
A. 01000001 B. 01000010 C. 01000011 D. 01000000
2. 以下关于 CSP 与 NOIP 的描述正确的是（ ）。
A. CSP 属于非专业认证，只有在校内才能参加
B. CSP 是中国电子学会举办的程序设计竞赛
C. CSP 和 NOIP 毫无关系，没参加 CSP 也可以直接参加 NOIP
D. CSP 和 NOIP 都是 CCF 旗下的程序设计赛事
3. 以下不能用作 C++ 程序中的标识符的是（ ）。
A. private B. friends C. news D. pascal
4. NOI 复赛测评机所用的 Linux 系统属于（ ）。
A. UML B. IDE C. OS D. Database
5. 如果 65 536 种颜色用二进制编码来表示，至少需要（ ）个二进制位。
A. 16 B. 8 C. 12 D. 10
6. 搜索算法中的 BFS 算法经常用到的数据结构是（ ）。
A. 堆 B. 栈 C. 链表 D. 队列
7. 在已经从小到大排好序的 n 元素单向链表中查询是否存在关键字为 k 的元素，最坏情况下运行的时间复杂度是（ ）。
A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log n)$
8. 在下列各种排序算法中，不是以“比较”作为主要操作的算法是（ ）。
A. 归并排序 B. 快速排序 C. 冒泡排序 D. 桶排序

9. 关于计算机网络,下面的说法中正确的是()。
- A. 现在的计算机必须连接到互联网才能正常运行
B. 192.168.0.1 是 A 类 IP 地址
C. 互联网的诞生用到了现代计算机技术和现代通信技术
D. 接入互联网的计算机的 IP 地址已经全部升级到了 IPv6 地址
10. 将(2, 6, 10, 17)分别存储到某个地址区间为 0~10 的哈希表中,如果哈希函数 $h(x) =$ (),将不会产生冲突,其中 $a\%b$ 表示 a 除以 b 的余数, sqrt 表示开平方, floor 表示向下取整。
- A. $x\%11$ B. $x^2\%11$ C. $2x\%11$ D. $\text{floor}(\text{sqrt}(x))\%11$
11. 现在有一个十六进制数 27,它等于二进制数的()。
- A. 100011 B. 100101 C. 100111 D. 100011
12. 以下逻辑表达式中,不管 A、B 如何取值,恒为假的是()。
- A. $(\neg A \vee B) \wedge (A \vee B) \wedge A$ B. $((\neg A \vee B) \vee (A \vee \neg B)) \wedge B$
C. $A \wedge ((\neg A \vee B) \vee (A \vee \neg B)) \wedge \neg A$ D. $((\neg A \vee B) \vee (A \vee \neg B)) \wedge A \wedge \neg B$
13. 某二叉树有 16 个结点都同时有左孩子结点和右孩子结点,则该二叉树中的叶子结点数是()个。
- A. 19 B. 17 C. 18 D. 16
14. 现有 16 张不同的卡片,其中红、黄、蓝、绿色卡片各 4 张。从中任取 3 张,要求红色最多有 1 张并且 3 张卡片不能是同一种颜色,不同的取法组合共有()种。
- A. 232 B. 472 C. 256 D. 484
15. 有 8 个结点的非连通无向图最多有()条边。
- A. 8 B. 7 C. 21 D. 49

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填√,错误填×;除特殊说明外,判断题每题 1.5 分,选择题每题 3 分,共计 40 分)

(1)

```
01 #include <bits/stdc++.h>
```

```
02 using namespace std;
03 int gcd(int a, int b){
04     int tmp;
05     if(b) tmp = a%b;
06     else return a;
07     while(tmp){
08         a = b;
09         b = tmp;
10         tmp = a%b;
11     }
12     return b;
13 }
14 int lcm(int a, int b){
15     return a/gcd(a,b)*b;
16 }
17 int main(){
18     int a,b;
19     cin >> a >> b;
20     cout << gcd(a, b) << endl;
21     return 0;
22 }
```

■ 判断题

16. 若输入 0 2024, 则输出结果为 0。 ()
17. 将第 5 行中的 if(b)改为 if(0 != b), 程序的运行结果不会改变。 ()
18. 若输入 2.4 4.8, 则输出错误。 ()
19. 将第 15 行 return a/gcd(a,b)*b 替换成 return a*b/gcd(a,b), 程序的运行结果不会改变。 ()

■ 选择题

20. 若输入数据为 20244204 12348, 则输出为 ()。
- A. 18 B. 36 C. 12 D. 24
21. 若将第 20 行 cout << gcd(a,b) << endl 替换成 cout << lcm(a, b) << endl, 输入数据为 20244204 12348, 则输出为 ()。
- A. 6,943,761,972 B. 程序出错, 无输出
- C. 3,471,880,986 D. 某个负数

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05     char s[128] = {0};
06     cin.getline(s,128);
07     for(int i = 0; i < strlen(s); ++i){
08         if(s[i] >= 65 && s[i] <= 90){
09             if(s[i] == 90)
10                 s[i] = 65;
11             else
12                 s[i] += 1;
13             s[i] ^= ' ';
14         }
15     }
16     cout << s << endl;
17     return 0;
18 }
```

■ 判断题

22. 输入一个长度大于 128 的字符串，程序的输出一定会出错。 ()
23. 将第 6 行 `cin.getline(s,128)` 更换为 `getline(cin,s)`，程序的运行结果不变。 ()
24. 将第 13 行 `s[i] ^= ' '` 更换为 `s[i] ^= 32`，程序的运行结果不变。 ()
25. 将第 9 行 `if(s[i] == 90)` 更换为 `if(s[i] == 'Z')`，程序的运行结果不变。 ()

■ 选择题

26. 若输入字符串 `s` 为 `CSPjs2024`，则输出为 ()。
- A. `dtqjs2024` B. `cspjs2024` C. `DTQjs2024` D. `CSPjs2024`
27. 若输出 `bcdea`，则输入字符串 `s` 为 ()。
- A. `BCDEA` B. `ABCDZ` C. `abcde` D. `bcdez`

(3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
```

```
03 int used[20],a[20],n;
04 long long ret=0;
05 bool flag;
06 void dfs(int x)
07 {
08     int i;
09     if(x > n)
10     {
11         flag=1;
12         for(i=1;i<=n;i++)
13             if(a[i]+i > n+2)
14             {
15                 flag=0;
16                 break;
17             }
18         if(flag) ret++;
19         return;
20     }
21     for(i=1;i<=n;i++)
22         if(used[i]==0)
23         {
24             used[i]=1, a[x]=i;
25             dfs(x+1);
26             used[i]=0, a[x]=0;
27         }
28 }
29 int main()
30 {
31     cin>>n;
32     dfs(1);
33     cout<<ret;
34     return 0;
35 }
```

■ 判断题

28. 如果输入 n 的值为 0, 那么程序在运行过程中一定会出现错误。 ()
29. 如果将第 26 行的 $a[x]=0$ 去掉, 输出的结果不会改变。 ()
30. 该程序算法的时间复杂度是 $O(n!*n)$ 。 ()
31. 输入某个正整数 n , 程序运行的输出结果可能会等于 0。 ()

■ 选择题

32. 若输入 $n=2$ ，那么输出结果是（ ）。
A. 1 B. 2 C. 3 D. 0
33. 若输入 $n=5$ ，那么输出结果是（ ）。
A. 16 B. 5 C. 10 D. 12
34. (4 分) 若输出结果为 128，则输入 n 是（ ）。
A. 8 B. 7 C. 16 D. 32

三、完善程序（单选题，每小题 3 分，共计 30 分）

- (1) 输入一个十进制正整数 n ，然后将 n 转换为二进制数，最后统计二进制数的各位数字，看看一共有多少位为 1，然后打印出总数。

输入格式：

第 1 行输入十进制正整数 n 。

输出格式：

输出一个整数，表示十进制正整数 n 转换成的二进制数中有多少位为 1。

输入样例：

127

输出样例：

7

样例说明：

十进制数 127 转换为二进制数 1111111，二进制位一共有 7 个 1，所以输出 7。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int a[33],cnt;
04 int main()
05 {
06     int n,sum,x;
07     cin>>n;
08     cnt=0;
09     ①;
10     while(x>0)
```

```

11  {
12      a[②]=x%2;
13      ③;
14  }
15  sum=0;
16  for(int i=1;④;i++)
17      if(a[i]==1)
18          ⑤;
19      cout<<sum<<endl;
20      return 0;
21  }

```

35. ①处应填 ()。

- A. $x=n$ B. $x=1$ C. $x=0$ D. $x=n-1$

36. ②处应填 ()。

- A. $--cnt$ B. $++cnt$ C. $cnt--$ D. cnt

37. ③处应填 ()。

- A. $x/=2$ B. $n++$ C. $x++$ D. $n--$

38. ④处应填 ()。

- A. $i<cnt$ B. $i<cnt/2$ C. $i<=cnt$ D. $i<=cnt/2$

39. ⑤处应填 ()。

- A. $sum--$ B. $sum=x$ C. $sum=0$ D. $sum++$

(2) 在一个 $n \times n$ 的棋盘上布满了 0 和 1, 如图(a)所示 ($n=7$)。为叙述方便, 将 0 用字母表示, 如图(b)所示。

	1	2	3	4	5	6	7
1	1	1	0	1	1	0	1
2	0	1	1	1	0	1	1
3	1	1	0	1	1	0	1
4	1	0	1	1	0	1	1
5	1	0	1	0	1	1	1
6	1	1	0	1	1	0	1
7	1	0	1	1	0	1	0

(a)

1	1	A	1	1	B	1
C	1	1	1	D	1	1
1	1	E	1	1	F	1
1	G	1	1	H	1	1
1	I	1	J	1	1	1
1	1	K	1	1	L	1
1	M	1	1	N	1	P

(b)

跳棋规则如下。

- (i) 从某个 0 格出发，可以向上、下、左、右 4 个方向连续越过若干个（至少 1 个）1 格后跳入下一个 0 格。如图(b)所示，从 A 出发，可跳到 B，或者跳到 E，但不能直接跳到 K。在跳到 B 之后还可以继续跳到 F，在跳到 E 之后可继续跳到 F 或 K，直到不能再跳为止。
- (ii) 每个 0 格只能到达一次，给出的起始点不能再次到达，也不能越过。跳过的距离为跳过 1 格的个数加 1，如从 A 到 B，跳过距离为 3，从 B 到 F，跳过距离为 2。

问题：当给出棋盘和起始点之后，最远能跳的距离是多少？

如图(b)所示，从 A 出发，可跳的路线不止一条，其中一条为：

A — B — F — L — K — E （可能不唯一）

3 2 3 3 3

它的跳过距离为 14。

输入格式：

第 1 行 3 个整数 n ($1 \leq n \leq 100$)、 x 、 y (x, y 是起始点坐标，图(b)中 A 处坐标为 1,3)。接下来 n 行，每行 n 个数（0 或 1），数与数之间用一个空格分隔。

输出格式：

一个整数，即最大可跳距离（若不能跳，则输出 0）。

输入样例：

```
4 3 2
1 0 1 0
1 1 1 1
0 0 1 0
1 1 0 1
```

输出样例：

6

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int n,i,j,k,ans;
04 int a[105][105],vis[105][105]; //a 表示棋盘，vis 统计点是否走过
05 int dx[4]={-1,1,0,0},dy[4]={0,0,1,-1};
06 void dfs(int x,int y,int step)
07 {
08     ans = ①;
```



```
09   for(int i=0;i<4;i++)
10   {
11       int tx=x,ty=y,s=0;
12       while(tx+dx[i]>0 && tx+dx[i]<=n
13           && ty+dy[i]>0 && ty+dy[i]<=n)
14       {
15           tx+=dx[i];
16           ty+=dy[i];
17           s++;
18           if(②)
19               break;
20       }
21       if(tx>0 && tx<=n && ty>0 && ty<=n && vis[tx][ty]==0
22           && a[tx][ty]==0 && s!=1)
23       {
24           vis[tx][ty]=1;
25           dfs(tx,ty,③);
26           ④;
27       }
28   }
29 int main()
30 {
31     int x,y,i,j;
32     cin>>n>>x>>y;
33     for(i=1;i<=n;i++)
34         for(j=1;j<=n;j++)
35             cin>>a[i][j];
36     ⑤;
37     dfs(x,y,0);
38     cout<<ans<<endl;
39     return 0;
40 }
```

40. ①处应填 ()。

A. 0

B. max(ans,step)

C. 1

D. step

41. ②处应填 ()。

A. `vis[tx][ty] == 1`

B. `vis[tx][ty] == 0`

C. `a[tx][ty] == 1`

D. `a[tx][ty] == 0`

42. ③处应填 ()。

A. `step+s`

B. `step+1`

C. `step`

D. `step-1`

43. ④处应填 ()。

A. `vis[tx][ty] = 1`

B. `vis[tx][ty] = 0`

C. `a[tx][ty] = 1`

D. `a[tx][ty] = 0`

44. ⑤处应填 ()。

A. `a[x][y] = 1`

B. `a[x][y] = 0`

C. `vis[x][y] = 1`

D. `vis[x][y] = 0`

普及组 CSP-J 2024 初赛模拟卷 2

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 在 C++ 程序中用到的一个常量 $a = 5e-6$ 在内存中占（ ）空间。
A. 2 字节 B. 1 字节 C. 4 字节 D. 8 字节
- 以下关于 CSP 与 NOIP 的描述正确的是（ ）。
A. CSP 属于专业认证，只有计算机专业在校生才能参加
B. CSP-J/CSP-S 是中国通信学会举办的程序设计竞赛
C. CSP-J 初赛零分也可以直接报名参加 NOIP
D. CSP-J 和 CSP-S 都是 CCF 牵头举办的程序设计赛事
- 某单位安装一条电信宽带进行上网，运营商说下行速度是 500 Mbps。要下载大小为 10 GB 的软件，最快大约需要（ ）秒。
A. 2 B. 20 C. 200 D. 2000
- 大写字母 M 的 ASCII 码整数值和空格的 ASCII 码整数值之和，是字母 m 的 ASCII 码整数值。空格的 ASCII 码整数值是（ ）。
A. 32 B. 31 C. 30 D. 29
- 在微型计算机中，（ ）的存取速度最快。
A. RAM B. CD-ROM C. 高速缓存 D. 寄存器
- 搜索算法中的 DFS 算法经常用到的数据结构是（ ）。
A. 堆 B. 栈 C. 链表 D. 队列
- 以下哪个说法是正确的？（ ）
A. 花括号“{”和“}”只能作为 C++ 函数体的定界符
B. 构成 C++ 程序的基本单位是函数，所有函数名都可以由用户命名
C. 分号是 C++ 语句之间的分隔符，不是语句的一部分
D. C++ 程序中的注释部分可以出现在程序中任意合适的地方

8. 在下列排序算法中, STL 中的 `sort()` 函数采用的主要算法是 ()。
- A. 选择排序 B. 快速排序 C. 冒泡排序 D. 拓扑排序
9. 以下哪个说法是正确的? ()
- A. 第一台电子计算机 ENIAC 是基于集成电路的产物
- B. 计算机必须要同时有 IP 地址和域名才能接入互联网
- C. `david@163.com` 是一个正确的电子邮箱地址
- D. 手机上收到的短信, 里面的链接可以随意点击打开
10. 以下不能对二维数组 `a` 进行正确初始化的语句是 ()。
- A. `int a[2][3]={{1,2},{3,4},{5,6}};`
- B. `int a[][3]={{1,2},{0}};`
- C. `int a[2][3]={0};`
- D. `int a[][3]={1,2,3,4,5,6};`
11. 现在有一个八进制数 274, 其转换成的二进制数是 ()。
- A. 10111011 B. 10111101 C. 10111100 D. 10101100
12. 设 $A = \text{true}$, $B = \text{false}$, $C = \text{false}$, $D = \text{true}$, 以下逻辑运算表达式的值为假的是 ()。
- A. $((A \wedge B) \vee C) \wedge D$ B. $(A \vee B) \wedge (C \vee D)$
- C. $A \wedge ((B \vee C) \vee D)$ D. $(A \wedge (B \vee C)) \vee D$
13. 二叉树的中序序列为 ABCDEFGHD, 后序序列为 ABFHGEDC, 则其前序序列为 ()。
- A. CBADEGHF B. CBADEGFH
- C. CBDAEGFH D. CBADGEFH
14. 从班级中体育比较好的 12 人中选 5 人去参加运动会, 其中甲、乙、丙最多同时选两人, 不同的选法共有 () 种。
- A. 792 B. 756 C. 720 D. 676
15. 以下哪个结构可以用来存储图? ()
- A. 栈 B. 二叉树 C. 邻接表 D. 队列

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<iostream>
02 #include<cstring>
03 using namespace std;
04 char s1[1005],s2[1005];
05 int a[1005],b[1005],c[1005];
06 int main()
07 {
08     int la,lb,lc;
09     scanf("%s",s1);
10     scanf("%s",s2);
11     la = strlen(s1);
12     lb = strlen(s2);
13     lc = max(la,lb) + 1;
14     for(int i=0;i<la;++i)
15         a[la-i] = s1[i] - '0';
16     for(int i=0;i<lb;++i)
17         b[lb-i] = s2[i] - '0';
18     for(int i=1;i<=lc;i++)
19     {
20         c[i] += a[i] + b[i];
21         c[i+1] = c[i]/10;
22         c[i] = c[i]%10;
23     }
24     if(c[lc]==0 && lc>0)
25         lc--;
26     for(int i=lc;i>0;i--)
27         printf("%d",c[i]);
28     return 0;
29 }
```

■ 判断题

16. 将第 2 行代码改为#include<stdio.h>，程序的运行结果不会改变。 ()
17. 将第 9~10 行代码改为 cin>>s1>>s2;，程序的运行结果不会改变。 ()
18. 若输入两个都超过 1005 位长的正整数，则程序一定会出错且无输出。 ()

19. 在输入 0 0 的情况下, 将第 24 行代码中的 `lc>0` 去掉, 程序的运行结果不会改变。

()

■ 选择题

20. 若输入数据为 1024 1000, 则输出为 ()。

- A. 24 B. 2024 C. 1024 D. 1000

21. 若输入数据为 1 -1, 则输出为 ()。

- A. 1 B. 0 C. -1 D. 以上都不是

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 10005;
04 int n, a[MAXN], b[MAXN];
05 void mergesort(int *a, int l, int r)
06 {
07     int i, j, cnt, mid;
08     if (l == r)
09         return;
10     mid = (l + r) / 2;
11     mergesort(a, l, mid);
12     mergesort(a, mid + 1, r);
13     i = l, j = mid + 1, cnt = 0;
14     while (i <= mid && j <= r)
15     {
16         if (a[i] <= a[j])
17             b[++cnt] = a[i++];
18         else
19             b[++cnt] = a[j++];
20     }
21     while (i <= mid)
22         b[++cnt] = a[i++];
23     while (j <= r)
24         b[++cnt] = a[j++];
25     for (i = l; i <= r; i++)
26         a[i] = b[i - l + 1];
27 }
```

```
28 int main(void)
29 {
30     cin >> n;
31     for (int i = 1; i <= n; i++)
32         cin >> a[i];
33     mergesort(a, 1, n);
34     for (int i = 1; i <= n; i++)
35         cout << a[i] << (i == n ? '\n' : ' ');
36     return 0;
37 }
```

■ 判断题

22. 该排序算法用到的是不稳定的排序算法。 ()
23. 将第 10 行改为 `mid = l + r >> 1;`, 程序的输出结果不变。 ()
24. 该排序算法用到了分治的思想。 ()
25. 第 35 行代码用到的三目运算符处理代码可以用等价的条件语句来写。 ()

■ 选择题

26. 在最坏情况下, 该算法的时间复杂度和下面哪个算法相当? ()
- A. 插入排序 B. 选择排序 C. 堆排序 D. 快速排序
27. 若输出 2 3 5 7 8, 则输入可能为 ()。
- A. 1 2 4 6 7 B. 8 7 5 2 3 C. 3 4 2 5 7 D. 8 2 3 4 5

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int t,x[100],a[100];
04 void dfs(int d,int i,int n)
05 {
06     if(n==1)
07     {
08         for(int k=0;k<d;k++)
09             printf("%4d",a[k]);
10         printf("\n");
11     }
12     else
13         for(int k=i;k<t;k++)
```

```
14         if(n%x[k]==0)
15         {
16             a[d]=x[k];
17             dfs(d+1,k,n/x[k]);
18         }
19     }
20 int main()
21 {
22     int n;
23     cin>>n;
24     for(int i=n;i>1;i--)
25         if(n%i==0)
26             x[t++]=i;
27     dfs(0,0,n);
28     return 0;
29 }
```

■ 判断题

28. 该程序的作用是对 n 进行质因数分解并从小到大依次打印。 ()
29. 将第 9 行代码 `printf("%4d",a[k]);` 中的 4 去掉, 程序输出不变。 ()
30. 第 24~26 行的作用是求出 n 的所有因子。 ()
31. 程序运行过程中, 若输入 n 为 0 或者负数, 程序一定会打印错误, 崩溃退出。 ()

■ 选择题

32. 若输入 6, 则输出为 ()。
- | | | | |
|------|-------|------|-------|
| A. 6 | B. 72 | C. 6 | D. 72 |
| 3 2 | 36 2 | 2 3 | 2 36 |
33. 若输入 $n=1$, 那么输出结果可能是 ()。
- | | | | |
|------|------|------|-----------|
| A. 2 | B. 1 | C. 0 | D. 什么也不输出 |
|------|------|------|-----------|
34. (4 分) 若输入 2024, 则输出有 () 行。
- | | | | |
|-------|-------|-------|-------|
| A. 18 | B. 20 | C. 21 | D. 19 |
|-------|-------|-------|-------|

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) 扫雷游戏是一款十分经典的单机小游戏。在 n 行 m 列的雷区中有一些格子含有地雷（称为地雷格），其他格子不含地雷（称为非地雷格）。玩家翻开一个非地雷格时，该格子中将会出现一个数字，提示周围格子中有多少个是地雷格。玩家的目标是在不翻出任何地雷格的条件下，找出所有的非地雷格。请将程序补充完整。

现在给出 n 行 m 列的雷区中的地雷分布，要求计算出每个非地雷格周围的地雷格数。

注：一个格子的周围格子包括其上、下、左、右、左上、右上、左下、右下这 8 个方向上与之直接相邻的格子。

输入格式：

第 1 行是用一个空格隔开的两个整数 n 和 m ，分别表示雷区的行数和列数。接下来 n 行，每行 m 个字符，描述了雷区中的地雷分布情况。字符*表示相应格子是地雷格，字符?表示相应格子是非地雷格。相邻字符之间无分隔符。

输出格式：

输出文件包含 n 行，每行 m 个字符，描述整个雷区。用*表示地雷格，用周围的地雷个数表示非地雷格。相邻字符之间无分隔符。

输入样例：

```
3 3
*??
???
?*?
```

输出样例：

```
*10
221
1*1
```

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int dx[] = {1, 1, 1, 0, 0, -1, -1, -1};
04 const int dy[] = {-1, 0, 1, -1, 1, -1, 0, 1};
05 char g[101][101];
06 int main()
07 {
08     int n,m,cnt;
09     cin>>n>>m;
```

```
10  for(int i=0;i<n;i++)
11      for(int j=0;j<m;j++)
12          cin>>g[i][j];
13  for(int i=0;i<n;i++)
14  {
15      for(int j=0;j<m;j++)
16          if ( ① )
17          {
18              ②;
19              for (int k = 0; ③ ; k++)
20                  if ( ④ )
21                      cnt++;
22              cout << cnt;
23          }
24          else
25              cout<<"*";
26          if( ⑤ )
27              cout<<endl;
28  }
29  return 0;
30 }
```

35. ①处应填 ()。

A. $g[i][j] \neq '?'$

B. $g[i][j] \neq '\backslash 0'$

C. $g[i][j] \neq '*'$

D. $g[i][j] == '*'$

36. ②处应填 ()。

A. $cnt++$

B. $cnt = 0$

C. $cnt = 0$

D. $++cnt = 0$

37. ③处应填 ()。

A. $k < 8$

B. $k < m$

C. $k < n$

D. $k < \min(m, n)$

38. ④处应填 ()。

A. $g[i + dy[k]][j + dx[k]] == '*'$

B. $g[i - dx[k]][j - dy[k]] == '*'$

C. $g[i + dx[k]][j + dy[k]] == '*'$

D. $g[i - dy[k]][j - dx[k]] == '*'$

39. ⑤处应填 ()。

A. $j \neq m$

B. $j \neq m-1$

C. $i \neq n$

D. $i \neq n-1$

(2) 给你 n 根火柴棍, 你可以拼出多少个形如 $A+B=C$ 的等式? 等式中的 A 、 B 、 C 是用火柴棍拼出的整数 (若该数非零, 则最高位不能是 0)。用火柴棍拼数字 0~9 的拼法如图所示。



注意:

1. 加号与等号各自需要两根火柴棍;
2. 如果 A 不等于 B , 则视 $A+B=C$ 与 $B+A=C$ 为不同的等式 ($A, B, C \geq 0$);
3. n 根火柴棍必须全部用上。

输入格式:

一个整数 n ($1 \leq n \leq 24$)。

输出格式:

一个整数, 表示能拼成的不同等式的数目。

输入样例:

18

输出样例:

9

样例说明:

9 个等式为 $0+4=4$ 、 $0+11=11$ 、 $1+10=11$ 、 $2+2=4$ 、 $2+7=9$ 、 $4+0=4$ 、 $7+2=9$ 、 $10+1=11$ 、 $11+0=11$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int hcb[10]={6,2,5,5,4,5,6,3,7,6};
04 int n;
05 int matches(int num)
06 {
07     int i,k=0;
08     for(i=num;i!=0;①)
09         ②;
```

```
10    if( ③ )
11        k+=hcb[0];
12    return k;
13 }
14 int main()
15 {
16     int i,j,count;
17     ④;
18     cin>>n;
19     for(i=0;i<=1000;i++)
20         for(j=0;j<=1000;j++)
21             if( ⑤ )
22                 count++;
23     cout<<count;
24     return 0;
25 }
```

40. ①处应填 ()。

- A. $i\%=10$ B. $i/=10$ C. $i++$ D. $i--$

41. ②处应填 ()。

- A. $k += hcb[i]$ B. $k += hcb[i/10]$
C. $k += hcb[i/10\%10]$ D. $k += hcb[i\%10]$

42. ③处应填 ()。

- A. $num==0$ B. $num!=0$ C. $num==n$ D. $num!=n$

43. ④处应填 ()。

- A. $count=1$ B. $count=match(n)$
C. $count=0$ D. $count=n$

44. ⑤处应填 ()。

- A. $matches(i)+matches(j)+matches(i+j)+6==n$
B. $matches(i)+matches(j)+matches(i+j)+3==n$
C. $matches(i)+matches(j)+matches(i+j)+4==n$
D. $matches(i)+matches(j)+matches(i+j)+5==n$

普及组 CSP-J 2024 初赛模拟卷 3

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 下列存储器按存取速度由快至慢排列，正确的是（ ）。
A. 硬盘 > 内存 > 高速缓存 > U 盘 B. 高速缓存 > 内存 > 硬盘 > U 盘
C. 高速缓存 > 硬盘 > 内存 > U 盘 D. U 盘 > 硬盘 > 内存 > 高速缓存
- 杨辉三角形和（ ）算法的思想最接近。
A. 贪心 B. 二分 C. DFS D. 递推
- 下列属于输入设备的是（ ）。
A. 显示器 B. 麦克风 C. 音箱 D. 打印机
- 小写字母 a 的 ASCII 码值为 97，小写字母 z 的 ASCII 码值是（ ）。
A. 120 B. 119 C. 122 D. 121
- IP 地址是每台上网的计算机所必需的，下列 IP 地址中可以作为合法主机地址的是（ ）。
A. 225.225.225.225 B. 200.256.192.8
C. 192.168.1.12 D. 0.0.0.0
- 下面哪个可以用作 C++ 程序中的标识符？（ ）
A. default B. private C. this D. them
- 快速排序在最坏情况下运行的时间复杂度是（ ）。
A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log n)$
- 字符串 a="98"，字符串 b="123"，使用 strcmp 函数，比较两者大小的结果是（ ）。
A. a 大 B. b 大 C. 一样大 D. 无法判断

9. 关于计算机网络,下面的说法哪个是正确的? ()
- A. 现在的计算机互联网是俄罗斯人发明的
 - B. 计算机网络拓扑结构只包括星形、流水线型和环形
 - C. TCP/IP 是因特网的最基本的协议
 - D. SMTP 属于物理层协议
10. 关于信息安全与网络道德,下列做法正确的是 ()。
- A. 确认环境安全后再输入支付密码
 - B. 随意点击不熟悉的电子邮件中的链接
 - C. 未经许可将其他人的私密照片和视频上传到互联网上
 - D. 在微信里随意转发未经证实的信息
11. 现在有一个十进制算式 $13 \times 64 + 7$, 它等于二进制数的 ()。
- A. 1100100111 B. 1101001111 C. 1101000111 D. 1111101101
12. 逻辑表达式 $A=\text{true}$, $B=C=D=\text{false}$, 则逻辑表达式取值为真的是 ()。
- A. $(A \wedge B) \vee (C \wedge D \vee \neg A)$ B. $\neg((A \wedge B \vee C) \wedge D)$
C. $A \wedge (B \vee C \vee D) \vee D$ D. $(A \vee (C \vee D)) \wedge B$
13. 某二叉树树根层次为 0, 则有 64 个结点的完全二叉树的高度是 ()。
- A. 9 B. 8 C. 7 D. 6
14. 书架上同一格放 5 本书, A 和 B 必须相邻, C 和 D 必须不相邻, 不同的放法共有 () 种。
- A. 24 B. 12 C. 18 D. 48
15. 字符串 $S = \text{"abcdefgh"}$ 的子串个数为 ()。
- A. 33 B. 35 C. 37 D. 36

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<bits/stdc++.h>
```

```
02 using namespace std;
03 const int SIZE = 100007;
04 int a[SIZE], b[SIZE];
05 int main()
06 {
07     int n, m, p, q, count=0, ret=0;
08     cin >> n >> m;
09     for (int i = 1; i <= n; i++)
10     {
11         cin >> p >> q;
12         a[p]++;
13         a[q+1]--;
14     }
15     for (int i = 1; i <= m; i++)
16     {
17         count += a[i];
18         ret += count;
19     }
20     cout << ret;
21     return 0;
22 }
```

注：输入流中 $1 \leq p \leq q < m$ 。

■ 判断题

16. 将输入的 p 和 q 改成任意的整数，运行程序都不会出错。 ()
17. 将第 7 行中的 $\text{count}=0$ 去掉，只定义 count 变量，程序的运行结果不会改变。 ()
18. 将第 17 行和第 18 行互换位置，程序的运行结果不会发生变化。 ()
19. 将第 15 行中的 $i = 1$ 改为 $i = 0$ ，程序的运行结果不会改变。 ()

■ 选择题

20. 将第 12~13 行中的 p 和 $q+1$ 分别改为 $p-1$ 和 q ，则输出结果 ()。
- A. 变大 B. 变小 C. 不变 D. 都有可能
21. 若输入为 4 4 1 2 2 3 3 3 1 3，则输出为 ()。
- A. 6 B. 10 C. 7 D. 8

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 bool fun(int n)
04 {
05     int i=7;
06     if(n==2 || n==3 || n==5)
07         return true;
08     if(n==1 || n%2==0 || n%3==0 || n%5==0)
09         return false;
10     while(i*i<=n)
11     {
12         if(n%i==0)
13             return false;
14         i += 4;
15         if(n%i==0)
16             return false;
17         i += 2;
18     }
19     return true;
20 }
21 int main()
22 {
23     int n,m;
24     cin>>n>>m;
25     if(fun(n) && fun(m) && fun(n+m+1))
26         cout<<"YES"<<endl;
27     else
28         cout<<"NO"<<endl;
29     return 0;
30 }
```

■ 判断题

22. 程序中 n 和 m 只有输入正整数, 程序的输出值才可能是 YES。 ()
23. 程序中用到了递归函数 `bool fun(int n)`。 ()
24. 若输入 n 和 m 都是素数, 程序的输出值一定是 YES。 ()
25. 若输入 n 和 m 的值分别是 -1 和 2027, 则程序的输出值是 YES。 ()

■ 选择题

26. 若输出 YES, 则输入可能为 ()。

- A. 23 29 B. 23 24 C. 23 27 D. 31 37

27. 若输出 NO, 则输入可能为 ()。

- A. 53 127 B. 2029 -1 C. 2023 2027 D. 97 41

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05     stack<int> s;
06     char a[1005];
07     int back,front,result;
08     cout << "Input: ";
09     cin >> a;
10     for (int i = 0; i < strlen(a); i++)
11     {
12         if (a[i] >= '0' && a[i] <= '9')
13             s.push(a[i] - '0');
14         else
15         {
16             back = s.top();
17             s.pop();
18             front = s.top();
19             s.pop();
20             if (a[i] == '+')
21                 result = front + back;
22             else if (a[i] == '-')
23                 result = front - back;
24             else if (a[i] == '*')
25                 result = front * back;
26             else if (a[i] == '/')
27                 result = front / back;
28             else if (a[i] == '%')
29                 result = front % back;
30             s.push(result);
31         }
32     }
```

```
33     cout << "Output: " << s.top() << endl;  
34     return 0;  
35 }
```

■ 判断题

28. 将第 1 行改为 `#include<iostream>`, 程序的运行结果不变。 ()
29. 本程序用到了队列而不是栈的思想。 ()
30. 将第 12 行中的 '0' 替换为 48, 程序的运行结果不会改变。 ()
31. 如果输入的都是非零数字和加、减、乘、除四则运算符号, 那么运行程序输出的值一定是正整数。 ()

■ 选择题

32. 本题的主要思想是求 () 表达式的值。
A. 前缀 B. 后缀 C. 中缀 D. 逻辑
33. 若输入 234--, 那么程序的输出结果是 ()。
A. 3 B. 2 C. 1 D. 0
34. (4 分) 若输入数据为 5432*%/, 则输出是 ()。
A. 3 B. 2 C. 1 D. 0

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 输入两个正整数 n 和 m ($1 < n < 10$, $1 < m < n$), 在 $1 \sim n$ 这 n 个数中任取 m 个数, 按字典序从小到大输出所有这样的排列。

输入格式:

第 1 行输入 n 和 m 。

输出格式:

输出从 n 个数中挑出 m 个数组成的所有排列, 按从小到大的顺序输出。

输入样例:

4 3

输出样例:

1 2 3

1 2 4

1 3 2

1 3 4
1 4 2
1 4 3
2 1 3
2 1 4
2 3 1
2 3 4
2 4 1
2 4 3
3 1 2
3 1 4
3 2 1
3 2 4
3 4 1
3 4 2
4 1 2
4 1 3
4 2 1
4 2 3
4 3 1
4 3 2

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 15;
04 bool flag, vis[MAXN];
05 int n,m,i,j,k,a[MAXN];
06 int main()
07 {
08     cin>>n>>m;
09     memset(vis,false,sizeof(vis));
10     for(i=1;i<=m;i++)
11     {
12         a[i]=i;
13         vis[i]=true;
14     }
15     flag=true;
```

```
16 while(flag)
17 {
18     for(i=1;i<=m-1;i++)
19         cout<<a[i]<<" ";
20     cout<<a[m]<<endl;
21     ① ;
22     for(i=m;i>=1;i--)
23     {
24         ② ;
25         for(j=a[i]+1;j<=n;j++)
26             if(!vis[j])
27             {
28                 vis[j]=true;
29                 ③ ;
30                 flag=true;
31                 break;
32             }
33     if(flag)
34     {
35         for(k=i+1;k<=m;k++)
36             for(j=1; ④ ;j++)
37                 if(!vis[j])
38                 {
39                     a[k]=j;
40                     vis[j]=true;
41                     break;
42                 }
43         ⑤ ;
44     }
45 }
46 }
47 return 0;
48 }
```

35. ①处应填 ()。

- A. flag = false
C. vis[1] = false

- B. flag = true
D. vis[1] = true

36. ②处应填 ()。

- A. vis[i] = true
C. vis[a[i]] = true

- B. a[i] = i
D. vis[a[i]] = false

37. ③处应填 ()。

- A. `a[i] = i` B. `a[i] = j` C. `a[i] = true` D. `a[i] = false`

38. ④处应填 ()。

- A. `j<=m` B. `j<=k` C. `j<=n` D. `j<=i`

39. ⑤处应填 ()。

- A. `exit` B. `return 0` C. `continue` D. `break`

(2) 求一个有向图中有多少个环并输出环的总数。

输入格式：

第 1 行为 n ，第 2 行为 n 个点的编号。

输出格式：

输出有向图的环的总数。

输入样例：

```
10
7 1 4 3 2 5 9 8 0 6
```

输出样例：

```
6
```

样例说明：

`a[0]=7,a[7]=8,a[8]=0` 构成一个环；`a[1]=1,{1}` 构成一个环；`a[2]=4,a[4]=2,{2,4}` 构成一个环；`a[3]=3,{3}` 构成一个环；`a[5]=5,{5}` 构成一个环；`a[6]=9,a[9]=6,{6,9}` 构成一个环。该有向图共有 6 个环。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int n, point[100];
04 bool vis[100];
05 int main()
06 {
07     int cnt;
08     scanf("%d", &n);
09     for (int i = 0; i < n; ++i)
10     {
11         scanf("%d", ①);
12         ②;
```

```
13     }
14     cnt = 0;
15     for (int i = 0; i < n; ++i)
16     {
17         if (③)
18         {
19             for (int j = i; !vis[j]; ④)
20             {
21                 vis[j] = true;
22             }
23             ⑤;
24         }
25     }
26     printf("%d", cnt);
27     return 0;
28 }
```

40. ①处应填 ()。

- A. &point B. point + i C. &point + i D. point[i]

41. ②处应填 ()。

- A. vis[j] = false B. vis[j] = true
C. vis[i] = true D. vis[i] = false

42. ③处应填 ()。

- A. !vis[i] B. vis[i]
C. !vis[point[i]] D. vis[point[i]]

43. ④处应填 ()。

- A. j = point[i] B. j = point[j]
C. i = point[j] D. i = point[i]

44. ⑤处应填 ()。

- A. cnt = j+1 B. cnt = n-j
C. ++cnt D. cnt = n-i

9. 关于计算机网络,下面的说法中哪个是正确的? ()
- A. 计算机网络是一个管理信息系统
B. 计算机网络是一个管理数据系统
C. 计算机网络是一个在协议控制下的多机互联系统
D. 计算机网络是一个独立的操作系统
10. 下列哪款软件不是操作系统软件的名字? ()
- A. 安卓 B. Windows 11 C. 华为鸿蒙 D. ChatGPT
11. 下述选项中哪个不是算法描述的通用方法? ()
- A. 自然语言 B. 流程图 C. 人工智能 D. 伪代码
12. 若 $A=\text{True}$, $B=\text{False}$, $C=\text{True}$, $D=\text{False}$, 以下逻辑运算表达式的运算结果为真的是 ()。
- A. $(A \wedge B) \vee (C \wedge D \vee \neg A)$ B. $((A \wedge B) \wedge C) \wedge \neg B$
C. $(B \vee C \vee D) \vee D \wedge A$ D. $(A \wedge (D \vee \neg C)) \wedge B$
13. 一棵二叉树的高度为 h , 所有结点的度数都为 0 或 2, 则此树最少有 () 个结点。
- A. 2^h-1 B. $2h-1$ C. $2h+1$ D. $h+1$
14. 从 12 个人中选出 5 个人, 其中甲、乙、丙必选的方法共有 () 种。
- A. 60 B. 36 C. 72 D. 120
15. 在一个有向图中, 所有顶点的入度之和等于所有顶点的出度之和的 () 倍。
- A. $1/2$ B. 2 C. 1 D. 4

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 char change(char str)
04 {
```



```
05     if(str >= 'a' && str <= 'z')
06         str -= 32;
07     return str;
08 }
09 int main()
10 {
11
12     string s1, s2;
13     cin >> s1 >> s2;
14     int cnt = 0;
15     for (int i = 0; i < s1.size(); i++)
16     {
17         for (int j = 0; j < s2.size(); j++)
18             if (change(s1[i]) == change(s2[j]))
19                 cnt++;
20     }
21     cout << cnt;
22     return 0;
23 }
```

■ 判断题

16. 将第 1 行头文件改为 `#include<iostream>`, 程序的运行结果不会改变。 ()
17. 将第 5 行中的 'a' 替换为 97, 程序的运行结果不会改变。 ()
18. 将第 6 行中的 32 替换为 ' ', 程序的运行结果不会改变。 ()
19. 将第 14 行代码去掉, 程序的运行结果不会改变。 ()

■ 选择题

20. 若输入数据为 ABCDE AbCdE, 则输出为 ()。
- A. 3 B. 5 C. 2 D. 0
21. 若输入数据为 WorldYiwuAsiaShanghai ChinaHangzhouZhejiangJinhua, 则输出为 ()。
- A. 36 B. 40 C. 42 D. 44

(2)

```
01 #include<iostream>
02 using namespace std;
```

```
03 int solve(int n,int m)
04 {
05     int i,sum;
06     if(m==1)
07         return 1;
08     sum=0;
09     for(i=1;i<n;i++)
10         sum += solve(i,m-1);
11     return sum;
12 }
13 int main()
14 {
15     int n,m;
16     cin>>n>>m;
17     cout<<solve(n,m)<<endl;
18     return 0;
19 }
```

■ 判断题

22. 如果 n 输入一个负整数, 程序的运行会出错。 ()
23. 如果 n 输入一个正整数, m 输入一个负整数, 那么程序会进入死循环, 不会输出任何结果。 ()
24. 若输入 4 4, 则程序的运行结果为 1。 ()
25. 若输入 4 -1, 则程序的运行结果为 0。 ()

■ 选择题

26. 若输入为 7 4, 则输出为 ()。
- A. 20 B. 10 C. 15 D. 5
27. 若输出为 10, 则输入可能为 ()。
- A. 5 3 B. 5 4 C. 6 4 D. 6 5

(3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 2e5 + 5;
04 int nums[MAXN];
```

```
05 int left_bound(int n, int target) {
06     int left = 0, right = n - 1;
07     while (left <= right) {
08         int mid = (left + right) / 2;
09         if(nums[mid] < target)
10             left = mid + 1;
11         else
12             right = mid - 1;
13     }
14     if(left < n && nums[left] == target)
15         return left;
16     return -1;
17 }
18 int right_bound(int n, int target) {
19     int left = 0, right = n - 1;
20     while (left <= right) {
21         int mid = (left + right) / 2;
22         if(nums[mid] <= target)
23             left = mid + 1;
24         else
25             right = mid - 1;
26     }
27     if(right >= 0 && nums[right] == target)
28         return right;
29     return -1;
30 }
31 int main()
32 {
33     int n, c;
34     cin>>n>>c;
35     for(int i = 0; i < n; ++i)
36         cin>>nums[i];
37     sort(nums, nums + n);
38     long long ans = 0;
39     for(int i = 0; i < n; ++i) {
40         int left = left_bound(n, nums[i] + c);
41         int right = right_bound(n, nums[i] + c);
42         if(left != -1)
43             ans += right - left + 1;
44     }
```

```
45     cout<<ans<<endl;
46     return 0;
47 }
```

■ 判断题

28. 本段程序的算法用到了二分算法的思想。 ()
29. 将第 3 行中的 `const` 去掉, 程序的运行结果不变。 ()
30. 将第 14 行中的 `left < n` 去掉, 程序的运行结果不变。 ()
31. 将第 38 行中的 `long long` 替换为 `int`, 程序的运行结果不变。 ()

■ 选择题

32. 第 8 行的写法在某些时候会导致程序运行有问题, 最好换成写法 ()。
- A. `mid = (left + right) << 1`
B. `mid = left + (right - left)/2`
C. `mid = (left + right) >> 1`
D. `mid = (left + right) % 2`
33. 本程序的时间复杂度为 ()。
- A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log n)$
34. (4 分) 当输入
- ```
4 1
1 1 2 3
```
- 时, 程序的输出结果为 ( )。
- A. 1      B. 2      C. 3      D. 4

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 给定一棵树, 输出树的根 `root`、孩子结点最多的结点 `max` 以及它的孩子结点。

输入格式:

第 1 行输入  $n$  (结点数  $\leq 100$ ) 和  $m$  (边数  $\leq 200$ )。以下  $m$  行输入每行两个结点  $x$  和  $y$ , 表示  $y$  是  $x$  的孩子结点 ( $x, y \leq 1000$ )。

输出格式:

第 1 行是树根 `root`。第 2 行是孩子结点最多的结点 `max`。第 3 行是 `max` 的孩子结点。

输入样例：

```
8 7
4 1
4 2
1 3
1 5
2 6
2 7
2 8
```

输出样例：

```
4
2
6 7 8
```

```
01 #include<iostream>
02 using namespace std;
03 int n,m,tree[105]={0};
04 int main()
05 {
06 int i,x,y,root,maxroot,sum=0,j,Max=0;
07 cin>>n>>m;
08 for(i=1;i<=m;i++)
09 {
10 cin>>x>>y;
11 ① ;
12 }
13 for(i=1;i<=n;i++) //找出树的根
14 if(②)
15 {
16 root=i;
17 ③ ;
18 }
19 for(i=1;i<=n;i++) //找孩子结点最多的结点
20 {
21 sum=0;
22 for(j=1;j<=n;j++)
23 if(tree[j]==i)
24 sum++;
```

```
25 if(④)
26 {
27 Max=sum;
28 maxroot=i;
29 }
30 }
31 cout<<root<<endl<<maxroot<<endl;
32 for(i=1;i<=n;i++)
33 if(⑤)
34 cout<<i<<" ";
35 return 0;
36 }
```

35. ①处应填 ( )。

A. tree[y] = x

B. tree[x] = y

C. tree[y] = i

D. tree[x] = i

36. ②处应填 ( )。

A. tree[i] == 1

B. tree[i] == 0

C. tree[i] == 2

D. tree[i]

37. ③处应填 ( )。

A. break

B. continue

C. return 0

D. exit

38. ④处应填 ( )。

A. sum == Max

B. sum <= Max

C. sum > Max

D. sum < Max

39. ⑤处应填 ( )。

A. tree[i] != maxroot

B. tree[i] <= maxroot

C. tree[i] >= maxroot

D. tree[i] == maxroot

(2) 快速排序是一种高效的排序算法, 我们常用的 STL 函数 sort 就是采用快速排序思想实现的。如下代码是一个经典的快速排序过程, 输入一个整数  $n$ , 然后输入  $n$  个整数, 程序会按照从小到大的顺序将所有整数进行排序并输出。请将程序补充完整。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int a[1005];
04 void quickSort(int a[], int begin, int end)
05 {
06 int i,j,tmp;
07 if(begin >= end)
08 return;
09 ① ;
10 i = begin;
11 j = end;
12 while(i < j)
13 {
14 while(a[j] > tmp)
15 j--;
16 while(②)
17 i++;
18 if(i != j)
19 swap(a[i], a[j]);
20 }
21 ③ ;
22 ④ ;
23 quickSort(a,i+1,end);
24 }
25 int main()
26 {
27 int n;
28 cin>>n;
29 for(int i=1;i<=n;i++)
30 cin>>a[i];
31 ⑤ ;
32 for(int i=1;i<=n;i++)
33 cout<<a[i]<<" ";
34 return 0;
35 }
```

40. ①处应填 ( )。

A. tmp = a[begin]

B. tmp = a[i]

C. tmp = a[j]

D. tmp = a[end]

41. ②处应填 ( )。

A. `a[i] >= tmp && i < j`

B. `a[i] >= tmp && i > j`

C. `a[i] <= tmp && i > j`

D. `a[i] <= tmp && i < j`

42. ③处应填 ( )。

A. `swap(a[i],a[j])`

B. `swap(a[begin],a[i])`

C. `swap(a[begin],a[j])`

D. `swap(a[begin],a[end])`

43. ④处应填 ( )。

A. `quickSort(a,begin,i)`

B. `quickSort(a,begin,i-1)`

C. `quickSort(a,1,i)`

D. `quickSort(a,1,i-1)`

44. ⑤处应填 ( )。

A. `quickSort(a,1,n-1)`

B. `quickSort(a,0,n-1)`

C. `quickSort(a,1,n)`

D. `quickSort(a,0,n)`



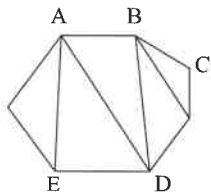
## 普及组 CSP-J 2024 初赛模拟卷 5



## 一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 十进制数 2024 的八进制表示是（ ）。  
A. 3749                      B. 3750                      C. 3751                      D. 3752
- 以下关于计算机协会竞赛的描述正确的是（ ）。  
A. NOI 国家集训队每年产生 4 名选手代表中国参加 IOI  
B. CSP-J/CSP-S 是 2018 年开始举办的  
C. USACO 晋级白金的选手可以直接参加 NOIP  
D. ACSL 和 NOIP 都是 CCF 旗下的程序设计赛事
- 以下哪个可以用作 C++ 程序中的变量名？（ ）  
A. public                      B. loops                      C. new                      D. delete
- 以下哪个数据结构不属于线性结构？（ ）  
A. 栈                      B. 数组                      C. 树                      D. 链表
- 以下哪个属于 STL 函数？（ ）  
A. main                      B. sort                      C. freopen                      D. scanf
- 小明用递归的方法写了一个斐波那契数列的程序，在这里递归函数经常用到的数据结构是（ ）。  
A. 树                      B. 栈                      C. 链表                      D. 队列
- 堆排序程序运行的时间复杂度是（ ）。  
A.  $O(\log n)$                       B.  $O(n)$                       C.  $O(n^2)$                       D.  $O(n \log n)$
- 在下列排序算法中，（ ）是稳定的排序算法。  
A. 归并排序                      B. 快速排序                      C. 选择排序                      D. 拓扑排序

9. 一台 32 位操作系统的计算机运行 C++, 下面哪个说法是正确的? ( )
- A. C++语言中的一个 int 类型的变量占 8 字节  
B. C++语言中的一个指针类型的变量占 4 字节  
C. C++语言中的一个 bool 类型的变量占 2 字节  
D. C++语言中的一个 double 类型的变量占 4 字节
10. 设全集  $I = \{a, b, c, d, e, f, g, h\}$ , 集合  $B \cup A = \{a, b, c, d, e, f\}$ ,  $C \cap A = \{c, d, e\}$ ,  $\sim B \cap A = \{a, d\}$ , 那么集合  $C \cap B \cap A$  为 ( )。
- A.  $\{c, e\}$                       B.  $\{d, e\}$                       C.  $\{e\}$                           D.  $\{c, d, e\}$
11. 在不大于 19 000 的正整数中, 与 19 000 互质的正整数有 ( ) 个。
- A. 9500                          B. 9498                          C. 9497                          D. 9499
12. 假设  $P=\text{true}$ ,  $Q=\text{false}$ ,  $R=\text{true}$ ,  $S=\text{true}$ , 逻辑运算表达式  $P \wedge Q \vee R \wedge S$  的值是 ( )。
- A. true                          B. false                          C. null                          D. NIL
13. 对于二叉树 T, 已知其前序遍历序列为 1 2 4 3 5 7 6, 中序遍历序列为 4 2 1 5 7 3 6, 则其后序遍历序列为 ( )。
- A. 4 2 5 7 6 3 1      B. 4 2 7 5 6 3 1      C. 4 2 7 5 3 6 1      D. 4 7 2 3 5 6 1
14. 一个口袋内装有大小相同的 7 个白球和 2 个黑球, 从口袋中取出 3 个球, 使其中不含黑球, 有多少种取法? ( )
- A. 32                          B. 35                          C. 24                          D. 56
15. 在下图中, 从顶点 ( ) 出发存在一条路径可以遍历图中的每条边一次, 而且仅遍历一次。
- A. B 点                          B. A 点                          C. E 点                          D. C 点



二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int a[100005];
04 bool judge(int x)
05 {
06 int i=2;
07 if(x==0 || x==1)
08 return false;
09 while(i <= floor(sqrt(x)) && (x%i != 0))
10 i++;
11 if(i > floor(sqrt(x)))
12 return true;
13 return false;
14 }
15 int inverted(int n)
16 {
17 int sum;
18 sum = 0;
19 while(n > 0)
20 {
21 sum = sum*10 + n%10;
22 n /= 10;
23 }
24 return sum;
25 }
26 int main()
27 {
28 int m,n,k;
29 bool flag;
30 k = 0;
31 flag = false;
32 cin >> m >> n;
33 for(int i=m; i<=n; i++)
34 if(judge(i) && judge(inverted(i)))
35 {
36 k++;
```

```
37 a[k] = i;
38 flag = true;
39 }
40 if(flag)
41 {
42 for(int i=1; i<k; i++)
43 cout << a[i] << ",";
44 cout << a[k] << endl;
45 }
46 else
47 cout<< "No" << endl;
48 return 0;
49 }
```

#### ■ 判断题

16. 将第 6 行中的 `i=2` 改为 `i=1`, 程序的运行结果不会改变。 ( )
17. 将第 9 行中的 `x%i != 0` 去掉, 程序的运行结果不会改变。 ( )
18. 将第 18 行删除, 程序的运行结果不会改变。 ( )
19. 将第 31 行删除, 程序的运行结果不会改变。 ( )

#### ■ 选择题

20. 若输入数据为 1949 2024, 则输出为 ( )。
- A. 1949,1987      B. 1949,1979      C. 1951,1979      D. 1951,1987
21. 若输出为 NO, 则输入可能为 ( )。
- A. 168 180      B. 785 792      C. 999 1020      D. 2024 2050

#### (2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05 string s;
06 int len,pos,i,j,sum;
07 sum = 0;
08 getline(cin,s);
09 len = s.size();
10 s[len] = ' ';
```

```
11 for(i=0; i<=len; i++)
12 {
13 if(s[i] != ' ')
14 sum++;
15 else
16 {
17 pos = i;
18 for(j=1; j<=sum; j++)
19 cout << s[--pos];
20 sum = 0;
21 if(i != len)
22 cout<<" ";
23 }
24 }
25 cout<<endl;
26 return 0;
27 }
```

### ■ 判断题

22. 将第 7 行删除, 程序的运行结果不会改变。 ( )
23. 将第 9 行中的 `s.size()` 改为 `s.length()`, 程序的运行结果不会改变。 ( )
24. 将第 10 行 `s[len] = ' '` 改为 `s[len] = 32`, 程序的运行结果不会改变。 ( )
25. 将第 20 行删除, 程序的运行结果不会改变。 ( )

### ■ 选择题

26. 若输入 CCF CSP, 则输出为 ( )。
- A. FCC PSC      B. CCF CSP      C. PSC FCC      D. PSC FCC
27. 将第 19 行中的 `--pos` 改为 `pos--`, 输入 CCF CSP, 则输出为 ( )。
- A. PS FC      B. CF SP      C. FC PS      D. CC SC

(3)

```
01 #include<bits/stdc++.h>
02 #define N 1000010
03 using namespace std;
04 int x, tot, a[N];
05 void calculate(int n, int step)
06 {
```

```
07 int ans;
08 ans = 1;
09 for(int i=1; i<=step-1; i++)
10 ans *= a[i];
11 if(ans > x)
12 return;
13 if(ans == x)
14 {
15 tot++;
16 return;
17 }
18 for(int i=a[step-1]; i<=x; i++)
19 if(n%i == 0)
20 {
21 n /= i;
22 a[step] = i;
23 calculate(n, step+1);
24 n *= i;
25 }
26 }
27 int main()
28 {
29 int n;
30 cin >> n;
31 while(n-->0)
32 {
33 tot = 0;
34 cin >> x;
35 a[0] = 2;
36 calculate(x, 1);
37 cout << tot << endl;
38 }
39 return 0;
40 }
```

#### ■ 判断题

28. 若将第 2 行替换为 `const int N=1000010;`, 程序的运行结果不会改变。 (     )
29. 若将第 8 行删除, 程序的运行结果不会改变。 (     )
30. 若将第 15 行中的 `tot++` 替换为 `++tot`, 程序的运行结果不会改变。 (     )
31. 将第 21 行和第 22 行交换, 程序的运行结果不会改变。 (     )

## ■ 选择题

32. 本程序中的算法用到了 ( ) 的思想。  
A. 贪心                      B. 搜索回溯                      C. 二分                      D. 动态规划
33. 若输入 2 24 36, 那么输出结果是 ( )。  
A. 7 9                      B. 7 8                      C. 8 9                      D. 8 8
34. (4 分) 若输入 2 96 2024, 那么输出结果是 ( )。  
A. 18 20                      B. 18 21                      C. 19 20                      D. 19 21

## 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 给定两个正整数  $l$  和  $r$ , 求区间  $[l, r]$  内素数的个数。如下代码是一个经典的计算过程, 请将程序补充完整。

输入格式:

第 1 行有两个整数, 分别代表询问次数  $n$  和给定区间的右端点最大值  $m$ 。接下来  $n$  行, 每行两个整数  $l$  和  $r$ , 代表一次查询。

输出格式:

对于每次查询输出一行, 若  $l, r \in [1, m]$ , 则输出区间内素数的个数, 否则输出 Crossing the line。

输入样例:

```
2 5
1 3
1 6
```

输出样例:

```
2
Crossing the line
```

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 1e6 + 5;
04 bool is_prime[MAXN];
05 int sum[MAXN];
06 void get_sum(int m)
07 {
08 memset(is_prime, true, sizeof(is_prime));
09 is_prime[0] = false;
```

```
10 is_prime[1] = false;
11 for(int i = 2; ①; ++i)
12 {
13 if(is_prime[i])
14 {
15 for(②; j <= m; j += i)
16 ③ ;
17 }
18 }
19 for(int i = 1; i <= m; ++i)
20 {
21 if(is_prime[i])
22 ④ ;
23 else
24 sum[i] = sum[i - 1];
25 }
26 }
27 int main()
28 {
29 int n, m, l, r;
30 cin>>n>>m;
31 get_sum(m);
32 while (n--)
33 {
34 cin>>l>>r;
35 if(l >= 1 && r <= m)
36 cout<< ⑤ <<endl;
37 else
38 cout<<"Crossing the line"<<endl;
39 }
40 return 0;
41 }
```

35. ①处应填 ( )。

- A.  $i \leq m$       B.  $i * i \leq m$       C.  $i \leq n$       D.  $i * i \leq n$

36. ②处应填 ( )。

- A. `int j = 1`      B. `int j = 2`      C. `int j = i`      D. `int j = i * i`

37. ③处应填 ( )。

- A. `is_prime[j] = true`      B. `is_prime[i] = true`  
C. `is_prime[j] = false`      D. `is_prime[i] = false`



38. ④处应填 ( )。

A. `sum[i]++`

B. `sum[i] += sum[i - 1]`

C. `sum[i] = sum[i - 1]`

D. `sum[i] = sum[i - 1] + 1`

39. ⑤处应填 ( )。

A. `sum[r+1] - sum[l]`

B. `sum[r+1] - sum[l-1]`

C. `sum[r] - sum[l-1]`

D. `sum[r] - sum[l]`

(2)  $N$  位同学站成一排, 音乐老师要请其中的  $(N-K)$  位同学出列, 使得剩下的  $K$  位同学排成合唱队形。合唱队形是指这样的一种队形: 设  $K$  位同学从左到右依次编号为  $1, 2, \dots, K$ , 他们的身高分别为  $T_1, T_2, \dots, T_K$ , 则他们的身高满足  $T_1 < \dots < T_i > T_{i+1} > \dots > T_K$  ( $1 \leq i \leq K$ )。

你的任务是, 已知所有  $N$  位同学的身高, 计算最少需要几位同学出列, 可以使得剩下的同学排成合唱队形。

输入格式:

输入的第 1 行是一个整数  $N$ , 表示同学的总数。第 2 行有  $N$  个整数, 用空格分隔, 第  $i$  个整数  $T_i$  是第  $i$  位同学的身高 (厘米)。

输出格式:

输出包括一行, 这一行只包含一个整数, 就是最少需要几位同学出列。

数据范围:  $2 \leq N \leq 100$ ,  $130 \leq T_i \leq 230$ 。

输入样例:

8

186 186 150 200 160 130 197 220

输出样例:

4

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 2024;
04 int n, ans=0;
05 int h[MAXN], f[MAXN], g[MAXN];
06 int main()
07 {
08 scanf("%d", &n);
09 for (int i = 1; i <= n; i++)
10 scanf("%d", &h[i]);
```

```
11 for (int i = 1; i <= n; i++)
12 {
13 ①;
14 for (int j = 1; j < i; j++)
15 if (②)
16 f[i] = max(f[i], f[j] + 1);
17 }
18 for (int i = n; i; i--)
19 {
20 g[i] = 1;
21 for (int j = n; ③ ; j--)
22 if (h[j] < h[i])
23 ④;
24 }
25 for (int i = 1; i <= n; i++)
26 ⑤;
27 printf("%d\n", n - ans);
28 return 0;
29 }
```

40. ①处应填 ( )。

- A.  $f[i] = 1$       B.  $f[i] = 0$       C.  $g[i] = 1$       D.  $g[i] = 0$

41. ②处应填 ( )。

- A.  $h[j] \leq h[i]$       B.  $h[j] < h[i]$   
C.  $h[j] \geq h[i]$       D.  $h[j] > h[i]$

42. ③处应填 ( )。

- A.  $j \geq i$       B.  $j \geq 0$       C.  $j > i$       D.  $j > 0$

43. ④处应填 ( )。

- A.  $g[i] = \max(f[i], f[j] + 1)$       B.  $g[i] = \max(f[i], g[j] + 1)$   
C.  $g[i] = \max(g[i], f[j] + 1)$       D.  $g[i] = \max(g[i], g[j] + 1)$

44. ⑤处应填 ( )。

- A.  $ans = \max(ans, f[i] + g[i] - 1)$       B.  $ans = \max(f[i], g[i] - 1)$   
C.  $ans = \max(ans, f[i] + g[i])$       D.  $ans = \max(g[i], f[i] - 1)$

## 普及组 CSP-J 2024 初赛模拟卷 6

扫码获取  
答案解析

## 一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 2024 的因子与质因子分别有（ ）个。  
A. 18 和 3              B. 16 和 3              C. 15 和 3              D. 16 和 4
- 使用邻接矩阵表示  $N$  个结点的有向图，所需要的存储空间为（ ）。  
A.  $N \times (N+1)$           B.  $N^2$                   C.  $N \times (N-1)$           D.  $N \times (N-1)/2$
- 在 C++ 程序中，表达式  $a\%=b$  与下列哪个表达式是等价的？（ ）  
A.  $a=\%b$               B.  $a/=b$                   C.  $a=b\%a$               D.  $a=a\%b$
- 线性表若采用链表存储结构，则要求内存中可用存储单元地址（ ）。  
A. 必须连续              B. 必须不连续  
C. 连续或不连续都行      D. 部分连续
- 我们输入一个新闻网站的网址便可访问该网站，其中用到的网络协议是（ ）。  
A. DNS                  B. FTP                      C. SSH                      D. TELNET
- 以下哪个不属于 STL 中栈的操作函数？（ ）  
A. empty                  B. front                      C. push                      D. pop
- 平面上任取  $n$  个整点（横坐标和纵坐标都是整数），其中一定存在两个点，它们的中点也是整点，那么  $n$  至少是（ ）。  
A. 4                      B. 5                      C. 6                      D. 7
- 以下哪个操作属于位运算范畴？（ ）  
A.  $\&\&$                       B.  $||$                       C.  $>>>$                       D.  $^$
- 关于树这种数据结构，下面的说法中哪个是正确的？（ ）  
A. 满二叉树的结点总数一定是奇数

- B. 完全二叉树的结点总数一定是奇数  
C. 树形结构只有双亲表示法和孩子表示法  
D. 二叉树的遍历方法只有前序遍历法和后序遍历法
10. 以下哪个选项不属于头文件 `cmath`? ( )  
A. `find(iterator first, iterator last, int x)`  
B. `abs(int x)`  
C. `ceil(double x)`  
D. `pow(double x, double y)`
11. 在 C++ 语言中, 表达式 `564|3` 的值等于 ( )。  
A. 7                      B. 5                      C. 4                      D. 3
12. 定义变量 `double n`, 如果下面的代码输入为 1000, 则输出最接近 ( )。  

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
 double n;
 cin>>n;
 cout << log10(n)-log2(n)<< endl;
 return 0;
}
```

  
A. 0                      B. -5                      C. -7                      D. 7
13. 在图的广度优先搜索中, 要维护一个标识数组表示已经访问过的图的结点, 需要 ( ) 数据结构存放结点以实现遍历。  
A. 栈                      B. 队列                      C. 哈希表                      D. 堆
14. 从一个  $6 \times 6$  的棋盘 (不可旋转) 中选取不在同一行也不在同一列的两个方格, 共有 ( ) 种方法。  
A. 480                      B. 450                      C. 360                      D. 720
15. 下列关于集合的说法哪个不正确? ( )  
A. 一个元素是否属于一个集合是确定的

- B. 集合中的元素两两不同
- C. 0 属于空集
- D. 集合中的元素不存在先后次序

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05 set <int> s;
06 srand(time(0));
07 int n=6;
08 int l=12;
09 int k,i=0,a[n];
10 while(s.size()<n)
11 {
12 k=rand()%10+1;
13 s.insert(k);
14 }
15 set <int>::iterator it;
16 for(it=s.begin(); it!=s.end(); it++)
17 a[i++] = *it;
18 for(int i=0; i<n; i++)
19 {
20 int x=rand()%n;
21 int y=rand()%n;
22 if(a[x] > a[y])
23 swap(a[x],a[y]);
24 }
25 for(int i=0; i<n; i++)
26 cout<<a[i]<<" ";
27 return 0;
28 }
```

■ 判断题

16. 若去掉第 6 行，程序的输出结果不受任何影响。

( )

17. 若去掉第 8 行, 程序的输出结果中的数字总数不变。 ( )
18. 程序的输出结果是一个从小到大排列的整数序列。 ( )
19. 将第 10 行中的 `s.size()` 替换成 `s.length()`, 程序的运行结果不会改变。 ( )

### ■ 选择题

20. 将第 13 行 `s.insert(k)` 替换成 `for(int i=1;i<=6;i++) s.insert(i)`, 则输出为 ( )。
- A. 1 2 3 4 5 6                      B. 6 5 4 3 2 1
- C. 1 2 3 6 5 4                      D. 1~6 随机分布值
21. 将第 13 行 `s.insert(k)` 替换成 `for(int i=1;i<=7;i++) s.insert(i)`, 第 20 行替换为 `int x=i+1`, 第 21 行替换为 `int y=i`, 则输出为 ( )。
- A. 1 2 3 4 5 6                      B. 6 5 4 3 2 1
- C. 2 3 4 5 6 7 1                      D. 2 3 4 5 6 7

### (2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int month[13]={-1,31,28,31,30,31,30,31,31,30,31,30,31};
04 int date,ans1,ans2,y,m,d;
05 bool check1(int date){
06 char s[32];
07 sprintf(s, "%d", date);
08 if(s[0]==s[7] && s[1]==s[6] && s[2]==s[5] && s[3]==s[4])
09 return true;
10 return false;
11 }
12 bool check2(int date){
13 char s[32];
14 sprintf(s, "%d", date);
15 if(s[0]==s[2] && s[0]==s[5] && s[0]==s[7] && s[1]==s[3]
 && s[1]==s[4] && s[1]==s[6])
16 return true;
17 return false;
18 }
19 int main()
20 {
21 cin>>date;
```

```
22 y=date/10000;
23 m=date/100%100;
24 d=date%100;
25 for(int i=y; ;i++){
26 if(i%400==0 || (i%100!=0 && i%4==0))
27 month[2] = 29;
28 else
29 month[2] = 28;
30 int j=(i==y)?m:1;
31 for(;j<=12;j++){
32 int k=(i==y && j==m)?d+1:1;
33 for(;k<=month[j];k++){
34 int date=i*10000+j*100+k;
35 if(check1(date) && ans1==0)
36 ans1=date;
37 if(check2(date))
38 return cout<<ans1<<" "<<date<<endl,0;
39 }
40 }
41 }
42 return 0;
43 }
```

注：输入为 8 位数字。

#### ■ 判断题

22. 将第 3 行中的 -1 改为 0，程序的运行不受任何影响。 ( )
23. 去掉第 15 行中的 `&& s[1]==s[4] && s[1]==s[6]`，程序的输出不变。 ( )
24. 将第 23 行 `m=date/100%100` 替换为 `m=date%10000/100`，程序的输出不变。 ( )
25. 将第 30 行改为 `int j; if(i==y && j==1 || i!=y && j==0) j=m`，程序的输出不变。 ( )

#### ■ 选择题

26. 若输入 20240204，则输出为 ( )。
- A. 20240204 20300302                      B. 20300302 20400402
- C. 20400402 21211212                      D. 20300302 21211212

27. 若输出 20011002 20200202, 则输入可能为 ( )。

- A. 20011002      B. 20001001      C. 20020101      D. 20020202

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 100;
04 bool Prime_Number_Judge(const int &num){
05 if (num <= 3)
06 return num > 1;
07 for (int i = 2; i < num; i++)
08 if (num % i == 0)
09 return false;
10 return true;
11 }
12 int& Get_Number_Size(const int &num){
13 int digit = 0, val = num;
14 while (val){
15 val /= 10;
16 digit++;
17 }
18 return digit;
19 }
20 vector<int>&Get_Digits(const int &num, vector<int> &digits){
21 int vactor_val = 0;
22 for(int num_size=Get_Number_Size(num);num_size>0;num_size--){
23 vactor_val = num % (int)pow(10.0, num_size);
24 vactor_val = vactor_val/((int)pow(10.0,num_size-1));
25 digits.push_back(vactor_val);
26 }
27 return digits;
28 }
29 vector<int>&Get_K_Adjacent(const int &num, vector<int>&adjacent){
30 vector<int> digists_number;
31 char tmp[128], buf_tmp[128];
32 Get_Digits(num , digists_number);
33 int digits = Get_Number_Size(num);
34 for (int i = 0; i < digits; i++){
35 for (int j = 0; j < digits-i; j++){
36 string buf;
```



```
37 int k = 0;
38 while (k <= i){
39 sprintf(tmp,"%d",digists_number.at(j+k));
40 buf += tmp;
41 k++;
42 }
43 for(int i=0;i<=buf.size();i++)
44 buf_tmp[i] = buf[i];
45 adjacent.push_back(atoi(buf_tmp));
46 }
47 }
48 return adjacent;
49 }
50 int main(){
51 int count=0;
52 for (int i = 1; i < MAXN; i++){
53 if (Prime_Number_Judge(i)){
54 vector<int> buf;
55 Get_K_Adjacent(i, buf);
56 int sign = 1;
57 for (int j = 0; j < buf.size(); j++){
58 if (!Prime_Number_Judge(buf.at(j))){
59 sign = 0;
60 break;
61 }
62 }
63 if (sign)
64 count++;
65 }
66 }
67 cout<<count;
68 return 0;
69 }
```

#### ■ 判断题

28. 若将第 5 行 `if (num <= 3)` 替换为 `if (num < 3)`, 程序的运行结果不会改变。 ( )
29. 若将第 7 行中的 `i < num` 替换为 `i*i <= num`, 程序的运行结果不会改变。 ( )
30. 本程序用到的 `vector` 属于 STL。 ( )
31. 若将 `MAXN=100` 替换为 `MAXN=70`, 程序的运行结果不会改变。 ( )



输出格式：

输出一行，包含  $n \times n$  个整数，由空格分隔，表示输入的矩阵经过 Z 字形扫描后的结果。

输入样例：

```
4
1 5 3 9
3 7 5 6
9 4 6 4
7 3 1 3
```

输出样例：

```
1 5 3 9 7 3 9 5 4 7 3 6 6 4 1 3
```

评测用例规模与约定：

$1 \leq n \leq 500$ ，矩阵元素为不超过 1000 的正整数。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05 int n,t,x,y;
06 t=1, x=1, y=1;
07 int v[505][505], p[505][505];
08 cin >> n;
09 for(int i=1;i<=n;i++)
10 for(int j=1;j<=n;j++)
11 cin>>p[i][j];
12 printf("%d ", ①);
13 ②;
14 while(③)
15 {
16 if(y+1 <= n)
17 {
18 ++t;
19 y++;
20 v[x][y] = 1;
21 printf("%d ",p[x][y]);
22 }
23 while(x+1<=n && y-1>=1 && ④)
24 {
```

```
25 ++t;
26 v[x+1][y-1] = 1;
27 printf("%d ",p[x+1][y-1]);
28 x++;
29 y--;
30 }
31 if(x+1<=n)
32 {
33 ++t;
34 x++;
35 v[x][y] = 1;
36 printf("%d ",p[x][y]);
37 }
38 while(x-1 >= 1 && y+1 <= n && !v[x-1][y+1])
39 {
40 ++t;
41 ⑤;
42 printf("%d ",p[x-1][y+1]);
43 x--;
44 y++;
45 }
46 }
47 return 0;
48 }
```

35. ①处应填 ( )。

- A. p[0][0]      B. p[0][1]      C. p[1][0]      D. p[1][1]

36. ②处应填 ( )。

- A. v[0][0] = 1    B. v[1][1] = 1    C. v[0][1] = 1    D. v[1][0] = 1

37. ③处应填 ( )。

- A. t < n\*n      B. t <= n\*n      C. t < n      D. t <= n

38. ④处应填 ( )。

- A. v[x+1][y-1]      B. v[x-1][y+1]  
C. !v[x+1][y-1]      D. !v[x-1][y+1]

39. ⑤处应填 ( )。

A.  $v[x-1][y-1] = 1$

B.  $v[x+1][y+1] = 1$

C.  $v[x+1][y-1] = 1$

D.  $v[x-1][y+1] = 1$

(2) 你有一架天平和  $N$  个砝码, 这  $N$  个砝码的重量依次是  $w_1, w_2, \dots, w_N$ 。请计算: 一共可以称出多少种不同的重量? 注意砝码可以放在天平两边。

输入格式:

输入的第 1 行包含一个整数  $N$ 。第 2 行包含  $N$  个整数:  $w_1, w_2, \dots, w_N$ 。

输出格式:

输出一个整数代表答案。

输入样例:

3  
1 4 6

输出样例:

10

样例说明:

能称出的 10 种重量是 1、2、3、4、5、6、7、9、10、11。

$1=1$

$2=6-4$  (天平一边放 6, 另一边放 4)

$3=4-1$

$4=4$

$5=6-1$

$6=6$

$7=1+6$

$9=4+6-1$

$10=4+6$

$11=1+4+6$

评测用例规模与约定:

对于 50% 的评测用例,  $1 \leq N \leq 15$ ;

对于所有评测用例,  $1 \leq N \leq 100$ ,  $N$  个砝码的总重量不超过  $10^5$ 。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int maxn = 105, maxv = 1e5 + 5;
```

```
04 int n, a[maxn], f[maxn][maxv], sum, ans;
05 int main()
06 {
07 cin >> n;
08 for(int i = 1; i <= n; i++)
09 {
10 cin >> a[i];
11 ①;
12 }
13 f[0][0] = 1;
14 for(int i = 1; i <= n; i++)
15 {
16 for(int ②; j >= 0; j--)//2
17 {
18 f[i][j] |= f[i - 1][j];
19 f[i][j] |= ③;
20 if(④)
21 f[i][j] |= f[i - 1][j + a[i]];
22 }
23 }
24 for(int i = 1; i <= sum; i++)
25 ⑤;
26 cout << ans;
27 return 0;
28 }
```

40. ①处应填 ( )。

A. sum += a[i]

B. sum += a[1]

C. sum += a[n]

D. sum = a[i]

41. ②处应填 ( )。

A. j = f[i][j]

B. j = sum-1

C. j = n

D. j = sum

42. ③处应填 ( )。

A. f[i-1][j-a[i]]

B. f[i-1][abs(j-a[i])]

C. f[i][abs(j-a[i])]

D. f[i-1][a[i]-j]

43. ④处应填 ( )。

A.  $j + a[i] < \text{sum}$

B.  $j + a[i] \leq \text{sum}$

C.  $j + a[i-1] < \text{sum}$

D.  $j + a[i-1] \leq \text{sum}$

44. ⑤处应填 ( )。

A.  $\text{ans} += f[n][i-1]$

B.  $\text{ans} += f[n-1][i-1]$

C.  $\text{ans} += f[n][i]$

D.  $\text{ans} += f[n-1][i]$

扫码获取  
答案解析

## 普及组 CSP-J 2024 初赛模拟卷 7

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在标准 ASCII 码表中，字符'4'的 ASCII 码值用二进制表示是（ ）。  
A. 00000100      B. 00110100      C. 00110101      D. 00110011
2. 关于树这种数据结构的下述说法，正确的是（ ）。  
A. 一个有  $m$  个顶点和  $m-1$  条边的图就是树  
B. 树中的任意两个顶点之间有且只有一条简单路径  
C. 树中有的结点可能构成环  
D. 若树根层次为 1，则对应高度为  $n$  的二叉树最多有  $2^n$  个结点
3. 以下哪个不是输入设备？（ ）  
A. 绘图仪      B. 触摸屏      C. 扫描仪      D. 麦克风
4. 当  $a=3$ ,  $b=2$ ,  $c=1$  时，执行以下程序段后  $c=$ （ ）。  

```
if(a>b)
 a=b;
if(b>c)
 b=c;
else
 c=b;
c=a--;
```

  
A. 0      B. 1      C. 2      D. 3
5. 学生在大学选修某些课程时需要先上其他的前置课程，所有课程和课程间的先修关系构成一个有向图  $G$ ，有向边  $\langle M, N \rangle$  表示课程  $M$  是课程  $N$  的先修课，则要找到某门课程  $L$  的全部先修课，下面哪种算法不可行？（ ）  
A. Dijkstra      B. BFS      C. DFS      D. BFS+DFS
6. 下列对语句 `freopen("function.in", "r", stdin);` 的分析中正确的是（ ）。



- A. freopen 是文件名  
B. function.in 是重定向函数名  
C. r 代表重定向为“写”方式  
D. 语句将 cin 重定向到文件 function.in
7. Windows 下可执行文件的扩展名是 ( )。  
A. com                      B. exe                      C. cpp                      D. dll
8. [x]补码=10011000, 其原码为 ( )。  
A. 011001111              B. 11101000              C. 11100110              D. 01100101
9. 下面有关布尔类型的函数的说法, 正确的是 ( )。  
A. 布尔类型函数只能返回 0 和 1 两个值  
B. 布尔类型函数可以返回负数  
C. 布尔类型函数必须有参数传递  
D. 布尔类型函数可以返回 NULL
10. 下面有关格雷码的说法, 错误的是 ( )。  
A. 在格雷码中, 任意两个相邻的代码只有一位二进制数不同  
B. 格雷码是一种可靠性编码  
C. 在格雷码中, 最大数和最小数只有一位二进制数不同  
D. 格雷码是一种唯一性编码
11. 现在有 5 个整数-2, -1, 0, 1, 2, 从中任意挑选两个整数, 它们的和为 0 的概率是多少?  
( )  
A. 1/6                      B. 1/4                      C. 1/5                      D. 1/10
12. 小明走楼梯, 每次上台阶能跨 1 或 2 级。下面是走到第  $N$  步台阶的 C++实现代码。  
该段代码采用的是 ( ) 算法。
- ```
int UpStairs(int N)
{
    if(N==1)
        return 1;
    else if(N==2)
        return 2;
```

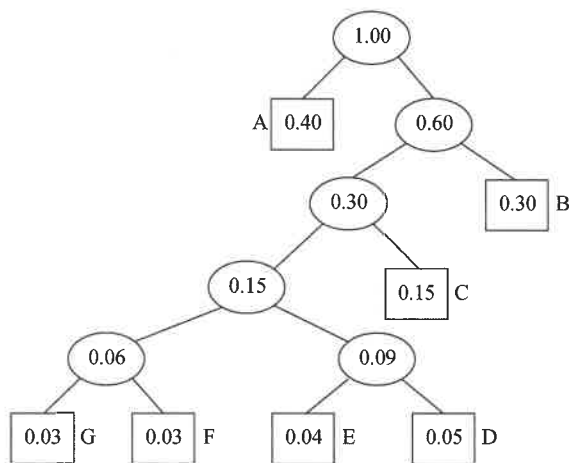
```

else
    return UpStairs(N-2) + UpStairs(N-1);
}

```

A. 递推 B. 贪心 C. 动态规划 D. 分治

13. 某内容中仅会出现 A, B, C, D, E, F, G, 对应的出现概率分别为 0.40, 0.30, 0.15, 0.05, 0.04, 0.03, 0.03, 如下图所示。按照哈夫曼编码规则, 假设 B 的编码为 11, 则 D 的编码为 ()。



A. 10010 B. 10011 C. 10111 D. 10001

14. 某学习小组有 5 名男生和 3 名女生, 从中选 3 名男生和 1 名女生参加 3 项竞赛活动, 每项活动至少有 1 人参加, 则参赛方法有 () 种。

A. 960 B. 1080 C. 2160 D. 540

15. 简单无向连通图 G 有 18 条边, 且每个顶点的度数为 2, 则图 G 有 () 个顶点。

A. 81 B. 17 C. 18 D. 64

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```

01 #include<bits/stdc++.h>
02 using namespace std;

```

```
03 int a[101000];
04 int p(int deep){
05     return pow(2,deep);
06 }
07 int main(){
08     int n,mindeep=1,i=2,deep=2;
09     scanf("%d %d",&n,&a[1]);
10     long long maxx = a[1];
11     for(i=2;i<=n;i++)
12         scanf("%d",&a[i]);
13     i=2;
14     while(i<=n){
15         long long sum=0;
16         for(;i<p(deep)&&i<=n;i++)
17             sum+=a[i];
18         if(sum > maxx)
19             maxx = sum, mindeep = deep;
20         deep++;
21     }
22     cout << mindeep;
23     return 0;
24 }
```

■ 判断题

16. 将第 8 行中的 `i=2` 改为 `i`, 程序的运行结果不会改变。 ()
17. 如果输入都是正整数, 那么将第 10 行中的 `a[1]` 替换成 `a[0]`, 将第 13 行中的 `i=2` 替换成 `i=1`, 程序的运行结果不会改变。 ()
18. 将第 15 行中的 `long long sum=0;` 替换成 `long long sum;`, 程序的运行结果不会改变。 ()
19. 将第 18 行中的 `if(sum > maxx)` 替换成 `if(sum >= maxx)`, 程序的运行结果不会改变。 ()

■ 选择题

20. 若输入为 7 1 6 5 4 3 2 1, 则输出为 ()。
- A. 11 B. 2 C. 10 D. 3
21. 若输入为 15 9 4 5 3 3 2 1 2 1 1 1 1 1 1, 则输出为 ()。
- A. 4 B. 3 C. 2 D. 1

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 bool judge(int a)
04 {
05     while(a!=0)
06     {
07         int t = a%10;
08         if(t==2 || t==4)
09             return 0;
10         a = a/10;
11     }
12     return 1;
13 }
14 int main(void)
15 {
16     int sum=0,n;
17     cin>>n;
18     for(int i=1;i<n/3+1;i++)
19         if(judge(i))
20             for(int j=i+1;j<n-i-j;j++)
21                 if(judge(j) && judge(n-i-j))
22                     {
23                         sum++;
24                     }
25     cout<<sum;
26     return 0;
27 }
```

■ 判断题

22. 将 judge 函数和 main 函数的位置调换一下, 程序的运行不受影响。 ()
23. 将第 3 行中的 bool 改为 int, 程序的运行结果不会改变。 ()
24. 该程序运行的时间复杂度是 $O(n\log n)$ 。 ()
25. 将第 20 行中的 $j=i+1$ 改为 $j=1$, 程序的运行结果不会改变。 ()

■ 选择题

26. 若输入 $n = 24$, 则程序的输出为 ()。
- A. 15 B. 16 C. 14 D. 13

27. 若程序的输出为 20, 则程序的输入可能为 ()。

A. 38

B. 41

C. 40

D. 42

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 typedef long long LL;
04 int a[100005],b[100005],c[100005];
05 int small(int n, int a[], int key)
06 {
07     int left=1, right=n;
08     while(left < right)
09     {
10         int mid = (left+right+1)/2;
11         if(a[mid] < key)
12             left = mid;
13         else
14             right = mid - 1;
15     }
16     return left;
17 }
18 int big(int n, int c[], int key)
19 {
20     int left=1, right=n;
21     while(left < right)
22     {
23         int mid = (left+right)/2;
24         if(c[mid] > key)
25             right = mid;
26         else
27             left = mid + 1;
28     }
29     return left;
30 }
31 int main()
32 {
33     int n;
34     cin>>n;
35     for(int i=1;i<=n;++i)
36         cin>>a[i];
```


34. (4 分) 若输入 3 1 2 3 3 4 5 5 6 7, 那么输出结果是 ()。

A. 27

B. 21

C. 18

D. 15

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 一个 $n \times n$ 的正方形的黑白瓦片的图案要被转换成新的正方形图案。写一个程序来找出将原始图案按照下列转换方法转换成新图案所采用的最小序号的方式。

①转 90° : 图案顺时针旋转 90° 。

②转 180° : 图案顺时针旋转 180° 。

③转 270° : 图案顺时针旋转 270° 。

④反射: 图案在水平方向翻转 (以中央铅垂线为中心形成原图案的镜像)。

⑤组合: 图案在水平方向翻转, 然后再按照①~③中的一种再次转换。

⑥不改变: 原图案不改变。

⑦无效转换: 无法用以上方法得到新图案。

如果有多种可用的转换方法, 请选择序号最小的那个。

只使用上述 7 种步骤中的一个来完成这次转换。

输入格式:

第 1 行一个正整数 n 。然后是 n 行, 每行 n 个字符, 全部为 @ 或 -, 表示初始的正方形。接下来的 n 行, 每行 n 个字符, 全部为 @ 或 -, 表示最终的正方形。

输出格式:

单独的一行包括 1~7 中的一个数字 (在上文已描述), 表明需要将转换前的正方形变为转换后的正方形的转换方法。

输入样例:

```
3
@-@
---
@@@
@-@
@--
--@
```

输出样例:

```
1
```

数据范围:

对于 100% 的数据, $1 \leq n \leq 10$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int n;
04 char a[15][15],b[15][15],c[15][15],d[15][15];
05 bool work1(){
06     for(int i=1;i<=n;i++)
07         for(int j=1;j<=n;j++)
08             ①;
09     for(int i=1;i<=n;i++)
10         for(int j=1;j<=n;j++)
11             if(b[i][j]!=c[i][j]) return 0;
12     return 1;
13 }
14 bool work2(){
15     for(int i=1;i<=n;i++)
16         for(int j=1;j<=n;j++)
17             ②;
18     for(int i=1;i<=n;i++)
19         for(int j=1;j<=n;j++)
20             if(b[i][j]!=c[i][j]) return 0;
21     return 1;
22 }
23 bool work3(){
24     for(int i=1;i<=n;i++)
25         for(int j=1;j<=n;j++)
26             ③;
27     for(int i=1;i<=n;i++)
28         for(int j=1;j<=n;j++)
29             if(b[i][j]!=c[i][j]) return 0;
30     return 1;
31 }
32 bool work4(){
33     for(int i=1;i<=n;i++)
34         for(int j=1;j<=n;j++)
35             ④;
36     for(int i=1;i<=n;i++)
37         for(int j=1;j<=n;j++)
38             if(b[i][j]!=c[i][j]) return 0;
39     return 1;
40 }
```



```
41 bool work5(){
42     ⑤;
43     for(int i=1;i<=n;i++)
44         for(int j=1;j<=n;j++)    a[i][j]=b[i][j];
45     if(work1())    return 1;
46     for(int i=1;i<=n;i++)
47         for(int j=1;j<=n;j++)    a[i][j]=b[i][j];
48     if(work2())    return 1;
49     for(int i=1;i<=n;i++)
50         for(int j=1;j<=n;j++)    a[i][j]=b[i][j];
51     if(work3())    return 1;
52     return 0;
53 }
54 bool work6(){
55     for(int i=1;i<=n;i++)
56         for(int j=1;j<=n;j++)
57             if(b[i][j]!=c[i][j]) return 0;
58     return 1;
59 }
60 void work(){
61     if(work1()) {cout<<1; return;}
62     if(work2()) {cout<<2; return;}
63     if(work3()) {cout<<3; return;}
64     if(work4()) {cout<<4; return;}
65     if(work5()) {cout<<5; return;}
66     if(work6()) {cout<<6; return;}
67     cout<<7;
68 }
69 int main(){
70     cin>>n;
71     for(int i=1;i<=n;i++)
72         for(int j=1;j<=n;j++)    {
73             cin>>a[i][j];
74             d[i][j]=a[i][j];
75         }
76     for(int i=1;i<=n;i++)
77         for(int j=1;j<=n;j++)    cin>>c[i][j];
78     work();
79     return 0;
80 }
```

35. ①处应填 ()。

A. $b[j][n-i+1]=a[j][i]$

B. $b[j][n-i]=a[j][i]$

C. $b[j][n-i+1]=a[i][j]$

D. $b[j][n-i]=a[i][j]$

36. ②处应填 ()。

A. $b[n-i][n-j]=a[i][j]$

B. $b[n-i+1][n-j+1]=a[i][j]$

C. $b[n-i][n-j]=a[j][i]$

D. $b[n-j+1][n-i+1]=a[i][j]$

37. ③处应填 ()。

A. $b[n-j+1][i]=a[i][j]$

B. $b[n-j][i]=a[i][j]$

C. $b[n-j+1][i]=a[j][i]$

D. $b[n-j][i]=a[j][i]$

38. ④处应填 ()。

A. $b[n-j+1][i]=a[i][j]$

B. $b[i][n-j]=a[i][j]$

C. $b[i][n-j+1]=a[i][j]$

D. $b[i][n-j]=a[j][i]$

39. ⑤处应填 ()。

A. `work1()`

B. `work2()`

C. `work3()`

D. `work4()`

(2) 一个旅行家想驾驶汽车以最少的费用从一个城市到另一个城市 (假设出发时油箱是空的)。给定两个城市之间的距离 D_1 、汽车油箱的容量 C (以升为单位)、每升汽油能行驶的距离 D_2 、出发点每升汽油的价格 P 、沿途加油站数 N (N 可以为零)、加油站 i 离出发点的距离 D_i 、每升汽油的价格 P_i ($i=1,2,\dots,N$)。计算结果四舍五入至小数点后两位。如果无法到达目的地, 则输出 No Solution。

输入格式:

第 1 行, D_1, C, D_2, P, N 。接下来有 N 行。第 $i+1$ 行, 两个数字, 即加油站 i 离出发点的距离 D_i 和每升汽油的价格 P_i 。

输出格式:

所需的最少费用, 计算结果四舍五入至小数点后两位。如果无法到达目的地, 则输出 No Solution。

输入样例:

275.6 11.9 27.4 2.8 2

102.0 2.9

220.0 2.2

输出样例:

26.95

说明/提示:

$N \leq 6$, 其余数字 ≤ 500 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 100;
04 int N;
05 double D[MAXN], P[MAXN], D1, C, per, req;
06 int main(){
07     cin>>D1>>C>>per>>P[0]>>N;
08     D1 /= per;
09     for(int i=1; i<=N; i++) {
10         cin>>D[i]>>P[i];
11         D[i] /= per;
12     }
13     D[0] = 0;
14     ①;
15     P[N+1] = 0;
16     double cur = 0, ans = 0;
17     for(int i=0; i<=N; i++) {
18         int ni = N+1;
19         for(int j=i+1; j<= N+1; j++) {
20             if(P[j] <= P[i]) {
21                 ni = j;
22                 break;
23             }
24         }
25         ②;
26         if(req > C)
27             ③;
28         if(req > cur) {
29             ans += (req - cur) * P[i];
30             cur = req;
31         }
32         ④;
33         if(cur < 0)
34             break;
```

```
35     }  
36     if(cur < 0)  
37         cout<<"No Solution"<<endl;  
38     else  
39         ⑤;  
40     return 0;  
41 }
```

40. ①处应填 ()。

A. $D[N] = D1$

B. $D[N+1] = D1$

C. $D[1] = D1$

D. $D[N-1] = D1$

41. ②处应填 ()。

A. $req = D[i] - D[i-1]$

B. $req = D[i+1] - D[i]$

C. $req = D[ni] + D[i]$

D. $req = D[ni] - D[i]$

42. ③处应填 ()。

A. $req = C$

B. $req -= C$

C. $req += C$

D. $req \% = C$

43. ④处应填 ()。

A. $cur += D[i+1] - D[i]$

B. $cur -= D[i+1] - D[i]$

C. $cur += D[ni] - D[i]$

D. $cur -= D[ni] - D[i]$

44. ⑤处应填 ()。

A. `printf("%.2d\n",ans)`

B. `printf("%2lf\n",ans)`

C. `cout<<fixed<<setprecision(2)<<ans<<endl`

D. `cout<<fixed<<setprecision(3)<<ans<<endl`

普及组 CSP-J 2024 初赛模拟卷 8

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 C++ 语言中，以下哪种方式不能用于向函数传递参数？（ ）
A. 值传递 B. 模板传递 C. 引用传递 D. 指针传递
2. 以下关于算法的描述中正确的是（ ）。
A. 算法是为解决问题而编制的计算机程序
B. 算法是为解决问题而采用的计算机程序设计语言
C. 算法是为解决问题而采用的计算方法
D. 算法是为解决问题而采取的方法与步骤
3. 在关系数据库中，存放在数据库中的数据逻辑结构以（ ）为主。
A. 二维表 B. 二叉树 C. 哈希表 D. 邻接表
4. 设某算法的时间复杂度函数的递推方程是 $T(n) = T(n-1) + n$ (n 为正整数) 及 $T(0) = 1$ ，则该算法的时间复杂度为（ ）。
A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log n)$
5. 在 C++ 语言中，一般提到的“空间复杂度”中的“空间”是指（ ）。
A. 程序运行时所占的内存大小 B. 程序运行时所占的数组大小
C. 程序运行时所占的硬盘大小 D. 程序源文件所占的硬盘大小
6. 2017 年 9 月 30 日是星期六，1999 年 9 月 30 日是（ ）。
A. 星期三 B. 星期六 C. 星期五 D. 星期四
7. 设栈 S 的初始状态为空，元素 1, 2, 3, 4, 5 依次入栈，以下出栈序列中不可能出现的是（ ）。
A. 1, 2, 3, 5, 4 B. 2, 3, 1, 5, 4 C. 1, 5, 3, 2, 4 D. 4, 3, 5, 2, 1
8. 关于分治算法，下列说法中错误的是（ ）。

- A. 分治算法的核心思想是分而治之，把问题转化为多个规模更小的子问题再求解
- B. 分治算法可以不使用递归实现
- C. 分治算法的时间复杂度是 $O(\log n)$ ，其中 n 表示问题的规模
- D. 二分法、快速排序等算法都是使用分治思想的典型算法

9. 如下代码主要表示什么数据结构？（ ）

```
typedef int LTDataType;
typedef struct ListNode
{
    struct ListNode* prev;
    struct ListNode* next;
    LTDataType data;
} LTNode;
```

- A. 单向链表 B. 双向链表 C. 循环链表 D. 优先队列

10. 使用如下代码片段定义 4 个字符串（假设头文件已正确定义），以下说法中错误的是（ ）。

```
string str1 = "csp";
string str2 = str1;
char str3[] = "csp";
char * str4 = str3;
```

- A. 对于 4 个字符串，都可以使用 `std::cout` 输出其中的内容（例如，`cout<<str3;`）
- B. `str3` 只占用 4 字节内存，但 `str1` 要占用更多内存
- C. 由于 `str2` 由 `str1` 直接赋值得到，因此二者指向同一块内存，即修改 `str1` 的内容后 `str2` 的内容也会随之改变
- D. 由于 `str4` 由 `str3` 直接赋值得到，因此二者指向同一块内存，即修改 `str3` 的内容后 `str4` 的内容也会随之改变

11. 冯·诺依曼体系结构中组成计算机的五大部件是（ ）。

- A. 总线、处理器、存储器、输入设备、输出设备
- B. 处理器、运算器、存储器、输入设备、输出设备
- C. 总线、控制器、存储器、输入设备、输出设备
- D. 控制器、运算器、存储器、输入设备、输出设备

12. 如下代码对树的操作是 ()。

```
void postorder(tree bt)
{
    if(bt)
    {
        postorder(bt->lchild);
        postorder(bt->rchild);
        cout << bt->data;
    }
}
```

- A. 后序遍历 B. 中序遍历 C. 前序遍历 D. 层次遍历
13. 有规格尺寸相同的 5 种颜色的手套 (不分左右) 各 15 只混装在箱子里, 从中至少取出多少只, 才能保证配出 3 副颜色相同的手套? ()
- A. 8 B. 9 C. 10 D. 11
14. 由 3 个 a、5 个 b 和 2 个 c 构成的字符串中, 包含子串 "abc" 的共有 () 个。
- A. 39 600 B. 780 C. 840 D. 260
15. 1 GB 代表的字节数量是 ()。
- A. 2^{10} B. 2^{20} C. 2^{30} D. 2^{40}

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int SIZE = 25;
04 int n,k,a[SIZE];
05 int cnt=0;
06 long long ans;
07 bool isPrime(int n)
08 {
09     cnt++;
10     cout<<cnt<<endl;
```

```
11     if(n<=1)
12         return false;
13     for(int i=2;i*i<=n;++i)
14         if(0==n%i)
15             return false;
16     return true;
17 }
18 void dfs(int m, int sum, int x)
19 {
20     if(m==k)
21     {
22         if(isPrime(sum))
23         {
24             ans++;
25         }
26         return;
27     }
28     for(int i=x;i<n;++i)
29         dfs(m+1,sum+a[i],i+1);
30     return;
31 }
32 int main()
33 {
34     cin >> n >> k;
35     for (int i = 0; i < n;++i)
36         cin >> a[i];
37     dfs(0,0,0);
38     cout << ans <<endl;
39     return 0;
40 }
```

■ 判断题

16. 将第 13 行中的 $i*i \leq n$ 改为 $i \leq \sqrt{n}$, 程序的运行结果不会改变。 ()
17. 将第 14 行中的 $if(0==n\%i)$ 改为 $if(n\%i==0)$, 程序的运行结果不会改变。 ()
18. 若输入 k 值比 n 大, 则程序会死循环, 无输出。 ()
19. 将第 29 行中的 $sum+a[i]$ 改为 $sum+=a[i]$, 程序的运行结果不会改变。 ()

■ 选择题

20. 若输入 k 的值为 6, 且输入 $n \geq k$, a 数组的元素均为相等的正整数, 则输出为 ()。
- A. 0 B. 2 C. 4 D. 8
21. 若输入 12 8, 则 `isPrime` 被调用的次数为 ()。
- A. 4 B. 66 C. 132 D. 495

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int CNT, p;
04 int check(int tmp, int n)
05 {
06     int res = 0;
07     for(int i=tmp; i>0; i--)
08     {
09         res += n;
10         int a=0, ans=res;
11         while(ans%n == 0)
12             a++, ans/=n;
13         i -= a-1;
14     }
15     return res;
16 }
17 int main()
18 {
19     cin >> CNT;
20     while(CNT--)
21     {
22         int ans=1;
23         cin >> p;
24         for(int i=2; i<=sqrt(p); i++)
25         {
26             int tmp=0;
27             while(p%i == 0)
28                 tmp++, p/=i;
29             if(tmp)
30                 ans = max(ans, check(tmp, i));
31         }
```

```
32     ans = max(ans,p);
33     cout<<ans<<endl;
34 }
35     return 0;
36 }
```

■ 判断题

22. 若输入 1 2, 程序的输出会报错。 ()
23. 若将第 27 行中的 while 改为 if, 程序的输出结果不变。 ()
24. 若输入 p 是一个素数, 则程序中的第 30 行不会被执行。 ()
25. 若输入 p 是一个素数, 则输出 ans 的值为 p。 ()

■ 选择题

26. 若输入 1 36, 则输出为 ()。
- A. 36 B. 6 C. 4 D. 9
27. 若 $\text{tmp} < n$, 则返回的 res 结果为 ()。
- A. n B. tmp C. $\text{tmp} * n$ D. $\text{tmp} + n$

(3)

```
01 #include<iostream>
02 #include<string>
03 #include<set>
04 using namespace std;
05 string a, s[105];
06 int cnt=0;
07 set<string>dict;
08 void trim(string &s)
09 {
10     if(!s.empty())
11     {
12         s.erase(0,s.find_first_not_of(" "));
13         s.erase(s.find_last_not_of(" ")+1);
14     }
15 }
16 int main()
17 {
```

```
18     cin>>a;
19     while(a!="#")
20     {
21         int length = a.length();
22         for(int i=0;i<length;i++)
23             if(isalpha(a[i]))
24                 a[i] = tolower(a[i]);
25         else
26             a[i] = ' ';
27         trim(a);
28         s[cnt]=a;
29         dict.insert(a);
30         cnt++;
31         cin>>a;
32     }
33     for(set<string>::iterator it=dict.begin();it!=dict.end();++it)
34         cout << *it << " ";
35     return 0;
36 }
```

■ 判断题

28. STL 中的 set 属于关联式容器。 ()
29. 若将第 6 行中的 `int cnt=0` 替换为 `int cnt`, 程序的运行结果不会改变。 ()
30. 若将第 21 行中的 `a.length()` 替换为 `a.size()`, 程序的运行结果不会改变。 ()
31. 若输入 34 12 56 #, 则程序输出 12 34 56。 ()

■ 选择题

32. 若输入为 A Dream Is To Build A Big Big World #, 则输出是 ()。
- A. A Dream Is To Build A Big Big World
- B. a dream is to build a big big world
- C. a big build dream is to world
- D. A Big Build Dream Is To World
33. 若输入为 WorldYiwu AsiaShanghai ChinaHangzhou ZhejiangJinhua #, 则输出是 ()。
- A. WorldYiwu AsiaShanghai ChinaHangzhou ZhejiangJinhua

- B.worldyiwu asiashanghai chinahangzhou zhejiangjinhua
 C.AsiaShanghai ChinaHangzhou WorldYiwu ZhejiangJinhua
 D.asiashanghai chinahangzhou worldyiwu zhejiangjinhua

34. (4分) 若输入为 I am 12 years old. #, 则输出是 ()。
- A.am i old years B.am I old years
 C.I am years old. D.I am 12 years old.

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 假设有一个无向图, 将这个无向图的某一条欧拉路或者欧拉回路用算法实现并打印出来。

输入格式:

第 1 行是 n 和 m , 表示有 n 个点和 m 条边, 以下 m 行描述每条边连接的两点。

输入样例:

```
5 5
1 2
2 3
3 4
4 5
5 1
```

输出格式:

欧拉路或欧拉回路。

输出样例:

```
1 5 4 3 2 1
```

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 #define MAXN 105
04 int g[MAXN][MAXN];
05 int du[MAXN];
06 int circuit[MAXN];
07 int n,e,circuitpos,x,y,start;
08 void find_circuit(int i)
09 {
10     for (int j = 1; j <= n; j++)
```

```
11     if ( ① )
12     {
13         g[j][i] = 0;
14         g[i][j] = 0;
15         find_circuit(j);
16     }
17     ②;
18 }
19 int main()
20 {
21     memset(g,0,sizeof(g));
22     cin >> n >> e;
23     for (int i = 1; i <= e; i++)
24     {
25         cin >> x >> y;
26         g[y][x] = 1;
27         ③;
28         du[x]++;
29         du[y]++;
30     }
31     start = 1;
32     for (int i = 1; i <= n; i++)
33         if ( ④ )
34             start = i;
35     circuitpos = 0;
36     ⑤;
37     for (int i = 1; i <= circuitpos; i++)
38         cout << circuit[i] << ' ';
39     cout << endl;
40     return 0;
41 }
```

35. ①处应填 ()。

A. `g[i][j] == 0`

B. `g[i][j] == 1`

C. `du[i] == 1`

D. `du[j] == 1`

36. ②处应填 ()。

A. `circuit[++circuitpos] = i`

B. `circuit[circuitpos] = i`

C. `circuit[j] = i`

D. `circuit[i] = j`

37. ③处应填 ()。

- A. start=1 B. start=0 C. g[x][y]=1 D. g[x][y]=0

38. ④处应填 ()。

- A. g[i][j] % 2 == 0 B. g[i][j] % 2 == 1
C. du[i] % 2 == 0 D. du[i] % 2 == 1

39. ⑤处应填 ()。

- A. find_circuit(0) B. find_circuit(start)
C. find_circuit(1) D. find_circuit(n)

(2) 设一棵 n 结点的二叉树 $tree$ 的中序遍历为 $(1, 2, 3, \dots, n)$, 其中数字 $1, 2, 3, \dots, n$ 为结点编号。每个结点都有一个分数 (均为正整数), 记第 i 个结点的分数为 d_i , $tree$ 及它的每棵子树都有一个加分, 任意一棵子树 $subtree$ (也包含 $tree$ 本身) 的加分计算方法如下:

$subtree$ 的左子树的加分 $\times subtree$ 的右子树的加分 $+ subtree$ 的根的分数

若某棵子树为空, 规定其加分为 1。叶子的加分就是叶子结点本身的分数, 不考虑它的空子树。试求一棵符合中序遍历为 $(1, 2, 3, \dots, n)$ 且加分最高的二叉树 $tree$ 。

要求输出: (1) $tree$ 的最高加分; (2) $tree$ 的前序遍历。

输入格式:

第 1 行是一个整数 n , 为结点个数。第 2 行是 n 个用空格隔开的整数, 为每个结点的分数 ($0 < \text{分数} < 100$)。

输出格式:

第 1 行是一个整数, 为最高加分 (结果不会超过 `int` 范围)。第 2 行是 n 个用空格隔开的整数, 为该树的前序遍历。如果存在多种方案, 则输出字典序最小的方案。

数据范围:

$n < 30$ 。

输入样例:

5
5 7 1 2 10

输出样例:

145
3 1 2 4 5

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int N=30;
04 int n, w[N], f[N][N], rt[N][N];
05 void print(int l, int r)
06 {
07     if(l > r)
08         return;
09     ①;
10     printf("%d ",root);
11     ②;
12     print(root+1, r);
13 }
14 int main()
15 {
16     memset(f,0,sizeof(f));
17     scanf("%d", &n);
18     for (int i=1; i<=n; ++i)
19     {
20         scanf("%d", &w[i]);
21         ③;
22         rt[i][i] = i;
23     }
24     for (int l=n; l>=1; l--)
25         for(int r=l+1; r<=n; r++)
26             for(int root=l; root<=r; root++)
27             {
28                 int lw=f[l][root-1];
29                 ④;
30                 if(lw == 0)
31                     lw = 1;
32                 if(rw == 0)
33                     rw = 1;
34                 if( ⑤ )
35                 {
36                     f[l][r] = lw*rw+w[root];
37                     rt[l][r] = root;
38                 }
39             }
40     printf("%d\n", f[1][n]);
```

```
41    print(1, n);  
42    printf("\n");  
43    return 0;  
44 }
```

40. ①处应填 ()。

A. `int root = 0`

B. `int root = 1`

C. `int root = rt[l][r]`

D. `int root = rt[1][n]`

41. ②处应填 ()。

A. `print(1, root-1)`

B. `print(l, root-1)`

C. `print(1, root)`

D. `print(l, root)`

42. ③处应填 ()。

A. `f[i][i] = 0`

B. `f[i][i] = 1`

C. `f[i][i] = i`

D. `f[i][i] = w[i]`

43. ④处应填 ()。

A. `int rw=f[root][r]`

B. `int rw=f[root+1][r]`

C. `int rw=f[root][n]`

D. `int rw=f[root+1][n]`

44. ⑤处应填 ()。

A. `lw*rw+w[root] > f[l][r]`

B. `lw*rw+w[root] < f[l][r]`

C. `lw*rw+w[root] > f[1][n]`

D. `lw*rw+w[root] < f[1][n]`

普及组 CSP-J 2024 初赛模拟卷 9

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 已知数组 A 中，每个元素 $A[i][j]$ 在存储时要占 3 字节，设 i 从 1 变化到 8， j 从 1 变化到 10，分配内存时是从地址 SA 开始连续按行存储分配的。试问： $A[4][7]$ 的起始地址是什么？（ ）

A. $SA+108$ B. $SA+141$ C. $SA+111$ D. $SA+126$

2. 以下关于算法的描述正确的是（ ）。

A. 算法可以没有输出
B. 算法至少有一个输入
C. 算法必须在计算机上用某种语言实现
D. 算法的改进在很大程度上推动了计算机科学与技术的进步

3. 执行下述 C++ 代码，输出的结果是（ ）。

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    int pos=2;
    string s("noip");
    cout<<s[++pos]<<endl;
    return 0;
}
```

A. o B. i C. p D. 空格

4. （ ）表示只读存储器。

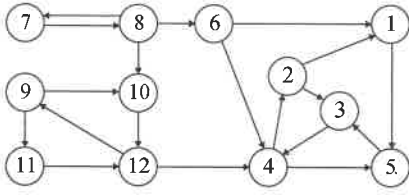
A. HDD B. SSD C. ROM D. RAM

5. 有关下面 C++ 代码的说法，正确的是（ ）。

```
int cnt;
int fun(int a, int b)
```

```
{  
    int tmp;  
    if(0==b) tmp=a;  
    else  
    {  
        tmp = fun(b,a%b);  
        cnt++;  
    }  
    return tmp;  
}
```

- A. fun 函数可以求出 a 和 b 的最大公共质因子
B. fun 函数可能会死循环
C. 如果 a 小于 9, 那么 cnt 的值不会超过 20
D. fun 函数能够求出 a 和 b 的最小公倍数
6. 以下哪个不属于与 STL 有关的函数? ()
A. swap B. sort C. max D. freopen
7. 在顺序表(1, 3, 7, 10, 14, 15, 19, 26, 38, 47, 85)中, 用二分法查找 13, 所需的关键码比较的次数为 ()。
A. 2 B. 3 C. 4 D. 5
8. 在下列各种排序算法中, 关键字比较的次数与记录的初始排列次序无关的是 ()。
A. 插入排序 B. 快速排序 C. 选择排序 D. 冒泡排序
9. 下列说法中正确的是 ()。
A. 计算机网络按照地理范围分为局域网、城域网和广域网
B. 互联网的基础 TCP/IP 是七层协议
C. 每台计算机配置的都是 C 类 IP 地址
D. 中国的计算机 IP 地址已经全部升级到了 IPv6 地址
10. 在 U 盘中发现计算机病毒后, 较为彻底的清除方法是 ()。
A. 删除 U 盘文件 B. 格式化 U 盘
C. 用消毒水喷洒在 U 盘上 D. 用杀毒软件处理

11. 下列关于十进制数 101 的说法中错误的是 ()。
- 注：这里 B 表示二进制 Binary，H 表示十六进制 Hexadecimal。
- A. 该数原码为 01100101 B B. 该数反码为 65 H
C. 该数真值为 01100011 B D. 该数补码为 65 H
12. 动态规划将一个问题分解为一系列子问题来求解。下面关于子问题的描述中正确的是 ()。
- A. 具有重叠子问题的性质
B. 和分治法的子问题类似
C. 不具有最优子结构的性质
D. 问题的最优解可以由部分子问题的非最优解推导出来
13. 在下面的有向图中，有多少个强连通分量？ ()
- 
- A. 3 B. 4 C. 5 D. 6
14. 从 4 台甲型电视机和 5 台乙型电视机中任意取出 3 台，其中至少要甲型电视机与乙型电视机各一台，则不同的取法共有 () 种。
- A. 140 B. 70 C. 80 D. 35
15. 在 32 位计算机中，用补码表示的 C++ 的整型变量 int 能够表示的数据范围是 ()。
- A. $0 \sim 2^{31} - 1$ B. $0 \sim 2^{32}$ C. $-2^{31} \sim 2^{31} - 1$ D. $-2^{31} + 1 \sim 2^{31}$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 priority_queue < int, vector<int>, greater<int> > q;
```

```
04 int main()
05 {
06     int n,x,ans=0,t1=0,t2=0,t3=0;
07     cin >> n;
08     for(int i=0; i<n; i++)
09     {
10         cin>>x;
11         q.push(x);
12     }
13     for(int i=n; i>1; i--)
14     {
15         t1 = q.top();
16         q.pop();
17         t2 = q.top();
18         q.pop();
19         t3 = t1+t2;
20         ans += t3;
21         q.push(t3);
22     }
23     cout<<ans<<endl;
24     return 0;
25 }
```

■ 判断题

16. priority_queue 是 STL 中的优先队列。 ()
17. 将第 3 行中的 greater<int>去掉, 不管输入如何, 程序的运行结果都不变。 ()
18. 将第 15 行和第 17 行互换, 不管输入如何, 程序的运行结果都不变。 ()
19. 将第 21 行去掉, 不管输入如何, 程序的运行结果都不变。 ()

■ 选择题

20. 若输入 3 1 2 9, 则输出 ans 为 ()。
- A. 12 B. 22 C. 23 D. 15
21. 若输入 4 5 4 3 6, 则输出 ans 为 ()。
- A. 30 B. 36 C. 37 D. 38

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 typedef long long LL;
04 LL fun(LL a, LL b, LL &x, LL &y)
05 {
06     LL t;
07     if(0==b)
08     {
09         x=1;
10         y=0;
11         return a;
12     }
13     LL d = fun(b,a%b,x,y);
14     t=x;
15     x=y;
16     y=t-(a/b)*y;
17     return d;
18 }
19 int main()
20 {
21     LL a,b,x=0,y=0;
22     cin >> a >> b;
23     cout << fun(a, b, x, y) << endl;
24     cout << x << " " << y << endl;
25     return 0;
26 }
```

■ 判断题

22. 将第 3 行中的 `typedef long long LL` 改为 `#define long long LL`, 程序的运行结果不会改变。 ()
23. 将第 4 行中的两个 `&` 去掉, 程序的运行结果不会改变。 ()
24. 若输入 $b=0$, 则输出结果是 a
 $0 \ 1$ 。 ()
25. 若输入两个为素数的正整数, 则输出结果中, x 和 y 一定有负数。 ()

■ 选择题

26. 若输入为 0 2024, 则输出为 ()。

- | | | | |
|---------|---------|------|------|
| A. 2024 | B. 2024 | C. 0 | D. 0 |
| 1 1 | 0 1 | 1 1 | 0 1 |

27. 若输入为 36 48, 则输出为 ()。

- | | | | |
|--------|--------|-------|-------|
| A. 144 | B. 144 | C. 12 | D. 12 |
| 1 -1 | -1 1 | 1 -1 | -1 1 |

(3)

```

01 #include <bits/stdc++.h>
02 using namespace std;
03 int a[101][3], b[101];
04 void dfs(int k, int w)
05 {
06     if(1==w)
07         cout<<k<<" ";
08     if(a[k][1]!=0)
09         dfs(a[k][1],w);
10     if(2==w)
11         cout<<k<<" ";
12     if(a[k][2]!=0)
13         dfs(a[k][2],w);
14     if(3==w)
15         cout<<k<<" ";
16 }
17 int main()
18 {
19     int n, m, t, root;
20     cin>>n;
21     for (int i=1; i<=n; i++)
22     {
23         cin>>t;
24         cin>>a[t][1]>>a[t][2];
25         b[a[t][1]]=1;
26         b[a[t][2]]=1;
27     }
28     for (int i=1; i<=n; i++)

```

```
29         if(b[i]==0)
30         {
31             root=i;
32             break;
33         }
34     dfs(root,1);
35     cout<<endl;
36     return 0;
37 }
```

■ 判断题

28. dfs 函数是处理二叉树的前、中、后序遍历的。 ()
29. 若将第 24 行 `cin>>a[t][1]>>a[t][2]` 更改为 `cin>>a[1][t]>>a[2][t]`, 程序的运行结果不会改变。 ()
30. 若将第 29 行去掉, 程序的运行结果不会改变。 ()
31. 若将第 34 行 `dfs(root,1)` 更换为 `dfs(root,4)`, 程序的运行结果不会改变。 ()

■ 选择题

32. 若输入如下数据, 则输出是 ()。

```
8
1 2 4
2 0 0
4 8 0
3 1 5
5 6 0
6 0 7
8 0 0
7 0 0
```

- | | |
|--------------------|--------------------|
| A. 3 1 2 4 8 5 6 7 | B. 2 1 8 4 3 6 7 5 |
| C. 2 8 4 1 7 6 5 3 | D. 3 2 1 8 4 5 7 6 |

33. 若输入和第 32 题一样, 将第 34 行 `dfs(root,1)` 更换为 `dfs(root,2)`, 则输出是 ()。

- | | |
|--------------------|--------------------|
| A. 3 1 2 4 8 5 6 7 | B. 2 1 8 4 3 6 7 5 |
| C. 2 8 4 1 7 6 5 3 | D. 3 2 1 8 4 5 7 6 |

34. (4分) 若输入和第32题一样, 将第34行 $\text{dfs}(\text{root}, 1)$ 更换为 $\text{dfs}(\text{root}, 3)$, 则输出是 ()。

A. 3 1 2 4 8 5 6 7

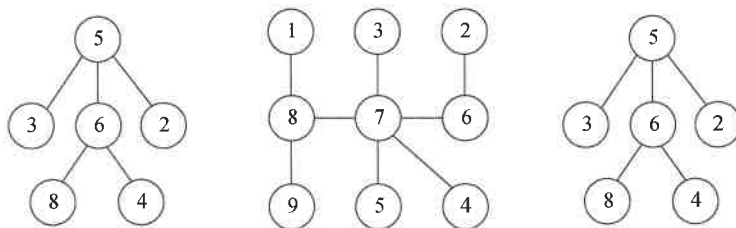
B. 2 1 8 4 3 6 7 5

C. 2 8 4 1 7 6 5 3

D. 3 2 1 8 4 5 7 6

三、完善程序 (单选题, 每小题3分, 共计30分)

(1) 米老鼠上次在唐老鸭的迷宫城堡里玩了很久, 现在她也想设计一个迷宫让唐老鸭来走。但是她设计迷宫的思路不一样, 她认为所有的通道都应该是双向连通的, 也就是说, 如果有一个通道连通了房间 A 和房间 B, 那么既可以通过它从房间 A 走到房间 B, 也可以通过它从房间 B 走到房间 A。为了提高难度, 米老鼠希望任意两个房间有且仅有一条路径可以相通 (除非走了回头路)。米老鼠现在把她的设计图给你, 让你帮忙判断她的设计图是否符合她的设计思路。比如下面的例子, 前两个是符合条件的, 但是最后一个有两种方法从 5 到达 8。



输入格式:

输入包含多组数据, 每组数据是一个以 0 0 结尾的整数对列表, 表示了一条通道连接的两个房间的编号。房间的编号至少为 1, 且不超过 100 000。每两组数据之间有一个空行。整个输入以两个 -1 结尾。

输出格式:

对于输入的每一组数据, 输出仅包括一行。如果该迷宫符合米老鼠的思路, 那么输出 YES, 否则输出 NO。

输入样例:

```
6 8 5 3 5 2 6 4 5 6 0 0
8 1 7 3 6 2 8 9 7 5 7 4 7 8 7 6 0 0
3 8 6 8 6 4 5 3 5 6 5 2 0 0
-1 -1
```


输出样例:

YES

YES

NO

```
01 #include<iostream>
02 using namespace std;
03 #define MAXN 100005
04 int fa[MAXN];
05 int visit[MAXN];
06 int path, node;
07 void init(int n){
08     for(int i=1;i<=n;++i)    {
09         fa[i] = i;
10         visit[i] = 0;
11     }
12     path = 0;
13     node = 0;
14 }
15 int get(int x){
16     if(x == fa[x])
17         return x;
18     else
19         return ①;
20 }
21 int main(){
22     int a,b;
23     bool flag;
24     cin>>a>>b;
25     while(a!=-1 && b!=-1){
26         init(MAXN);
27         flag = true;
28         if(0==a && 0==b)
29             path--;
30         while( ② ){
31             path++;
32             if(visit[a]==0){
33                 visit[a]=1;
34                 node++;
35             }
```

```
36         if(visit[b]==0){
37             visit[b]=1;
38             node++;
39         }
40         int root1=get(a);
41         int root2=get(b);
42         if( ③ )
43             flag = false;
44         else
45             ④;
46         cin>>a>>b;
47     }
48
49     if( ⑤ )
50         cout<<"NO"<<endl;
51     else
52         cout<<"YES"<<endl;
53     cin>>a>>b;
54 }
55 return 0;
56 }
```

35. ①处应填 ()。

- | | |
|-----------------------|------|
| A. fa[x] | B. x |
| C. fa[x] = get(fa[x]) | D. 1 |

36. ②处应填 ()。

- | | |
|-----------------|-----------------|
| A. a!=0 b!=0 | B. a!=0 && b!=0 |
| C. a!=0 | D. b!=0 |

37. ③处应填 ()。

- | | |
|-------------------|-------------------------|
| A. root1 == root2 | B. root1 != root2 |
| C. flag == true | D. visit[a] == visit[b] |

38. ④处应填 ()。

- | | |
|----------------------|----------------------|
| A. fa[root1] = root1 | B. fa[root2] = root2 |
| C. fa[root1] = root2 | D. flag = true |

39. ⑤处应填 ()。

- A. path != node && !flag B. path != node || !flag
C. path != node-1 && !flag D. path != node-1 || !flag

(2) 给定一个字符串, 请计算至少添加几个字符, 可以让其构成回文字符串。

输入样例:

5

acbcd

输出样例:

2

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 #define MAXN 5005
04 char str1[MAXN], str2[MAXN];
05 int maxlen[2][MAXN];
06 int main()
07 {
08     int n;
09     while(cin>>n)
10     {
11         for(int i=1;i<=n;++i)
12             cin>>str1[i];
13         str1[n+1]='\0';
14         for(int i=1; i<=n; ++i)
15             ①;
16         memset(maxlen,0,sizeof(maxlen));
17         int x=0;
18         for(int i=1; i<=n; ++i)
19         {
20             x = 1-x;
21             for(int j=0; j<=n; ++j)
22             {
23                 if(str1[i] == str2[j])
24                     ②;
25                 else
26                 {
27                     int len1 = ③;
```


普及组 CSP-J 2024 初赛模拟卷 10

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 用 C++ 语言将一个大写字母转换为对应的小写字母（比如将大写字母 A 转换为小写字母 a），以下哪个代码不正确？（ ）

A. `c+=32` B. `c-=32` C. `c^=' '` D. `c^=32`

2. 下列说法中错误的是（ ）。

A. 栈、队列、树和图都属于数据结构的范畴
B. CSP-J/CSP-S 是中国计算机学会举办的程序设计竞赛
C. 背包问题属于动态规划算法中的一种类型
D. 按照分布的地理范围，计算机网络类型可以分为星形、环形和总线型

3. 下列 C++ 数组的定义中，会丢失数据的是（ ）。

A. `char str1[] = {'C', 'S', 'P'};`
B. `int num1[] = {3, 6, 9};`
C. `char str2[] = {'CSP-J', 'CSP-S', 'NOIP'};`
D. `float num2[] = {1.0, 3.0, 5.0};`

4. 执行下面的 C++ 代码，输出是（ ）。

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    int tmp = 1;
    for(int i=1; i<10; i++)
        for(int j=1; j<5; j++)
            if(i%j == 1)
                tmp++;
    cout<<tmp;
    return 0;
}
```

A. 10 B. 11 C. 12 D. 13

5. 24 位真彩色能表示 () 种颜色。
A. 16 777 216 B. 256 C. 24 D. 65 536
6. 以下哪个不属于 STL 中队列 (queue) 的操作函数? ()
A. push B. pop C. empty D. top
7. 归并排序算法的空间复杂度是 ()。
A. $O(\log n)$ B. $O(n)$ C. $O(1)$ D. $O(n \log n)$
8. 下列 C++ 代码执行后不能输出 "NOIP" 的是 ()。
A. `string s("NOIP"); cout<<s<<endl;`
B. `string s="NOIP"; cout<<s<<endl;`
C. `string s("NOIP"); cout<<s[1]<<s[2]<<s[3]<<s[4]<<endl;`
D. `string s{"NOIP"}; cout<<s<<endl;`
9. 哈夫曼树的构造用到了 () 的算法思想。
A. 二分查找 B. 搜索回溯 C. 贪心 D. 动态规划
10. 在 C++ 语言中, 一个字符指针变量 `char *p` 占 () 字节。
A. 1 B. 2 C. 8 D. 4
11. 某数列有 2024 个各不相同的单元, 由低至高按序排列。现要对该数列进行二分搜索 (binary search)。在最坏情况下, 需搜索 () 个单元。
A. 2024 B. 11 C. 12 D. 1012
12. 在 C++ 程序中, 判断 `a` 不等于 0 且 `b` 不等于 0 且 `c` 不等于 0 的正确的条件表达式是 ()。
A. `!((a!=0) || (b!=0) || (c!=0))`
B. `!((a!=0) && (b!=0) && (c!=0))`
C. `a && b && c`
D. `(a=0) && (b=0) && (c=0)`
13. 某二叉树有 4 个结点, 则该二叉树的不同形态有 () 种。
A. 12 B. 14 C. 10 D. 16

14. 设含有 8 个元素的集合的全部子集数为 S, 其中由 6 个元素组成的子集数为 T, 则 T/S 的值为 ()。
- A. 7/16 B. 7/64 C. 7/32 D. 21/256
15. 对于给定的序列 $\{a_k\}$, 我们把 (i, j) 称为逆序对, 当且仅当 $i < j$ 且 $a_i > a_j$ 。那么序列 1, 9, 2, 4, 8, 6 的逆序对有 () 个。
- A. 7 B. 6 C. 5 D. 4

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int a[10];
04 void print_num(){
05     cout<<'1';
06     for(int i=2;i<=9;++i){
07         if(a[i]==0)
08             cout<<'+';
09         else
10             cout<<'-' ;
11         cout<<i;
12     }
13     cout<<"=9"<<endl;
14     return;
15 }
16 int main(){
17     for(int i=0; i<=255; i++){
18         int k=i, s = 1;
19         for(int j=9;j>=2;j--){
20             if(k&1){
21                 s-=j;
22                 a[j]=1;
23             }
24             else{
25                 s+=j;
26                 a[j]=0;
```

```
27         }
28         k>>=1;
29     }
30     if(s==9)
31         print_num();
32 }
33 return 0;
34 }
```

■ 判断题

16. 将第 8 行和第 10 行交换, 程序的运行结果不会改变。 ()
17. 将第 8 行中的 '+' 改为 "+", 程序的运行结果不会改变。 ()
18. 将第 20 行中的 $k\&1$ 改为 $k\&2$, 程序的运行结果不会改变。 ()
19. 将第 28 行中的 $k>>=1$ 改为 $k/=2$, 程序的运行结果不会改变。 ()

■ 选择题

20. 输出数据共有 () 行。
A. 11 B. 10 C. 12 D. 13
21. 程序主要用到了 () 的算法思想。
A. 贪心 B. 二分 C. 二进制 D. 动态规划

(2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 stack <char> s;
04 int main()
05 {
06     string a;
07     int len,mid,next,top;
08     getline(cin,a);
09     len=a.length();
10     mid=len/2-1;
11     for(int i=0; i<=mid; i++)
12         s.push(a[i]);
13     if(len%2==0)
14         next=mid+1;
15     else
```



```
16     next=mid+2;
17     for(int i=next;i<=len-1;i++)
18     {
19         if(a[i] != s.top())
20             break;
21         s.pop();
22     }
23     if(s.size()==0)
24         cout<<"yes";
25     else
26         cout<<"no";
27     return 0;
28 }
```

■ 判断题

22. 将第 6 行中的 string a 改为 char a[256], 程序的输出结果不变。 ()
23. 即使输入的字符串中间有空格, getline 函数也支持相应的处理。 ()
24. 将第 11 行中的 int i=0 改为 int i=1, 程序的输出结果不变。 ()
25. 若输入 abc cba, 同时将第 8 行的 getline(cin,a)改为 cin>>a, 程序的输出结果为 no。 ()

■ 选择题

26. 若输出结果为 yes, 则输入可能为 ()。
- A. abc big big abc B. abc big big cba
C. abc big gib abc D. abc big gib cba
27. 若输出结果为 no, 则输入可能为 ()。
- A. 1234567890987654321 B. 12345 6789899876 54321
C. 12345 678909876 54321 D. 12345 6789009876 54321

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 struct node
04 {
05     int data;
06     node *next;
```

```
07 };
08 int n,m;
09 node *head,*p,*r;
10 int main()
11 {
12     cin >> n >> m;
13     head = new node;
14     head->data = 1;
15     head->next = NULL;
16     r = head;
17     for(int i=2;i<=n;i++)
18     {
19         p = new node;
20         p->data = i;
21         p->next = NULL;
22         r->next = p;
23         r = p;
24     }
25     r->next = head;
26     r = head;
27     for(int i=1;i<=n;i++)
28     {
29         for(int j=1;j<=m-2;j++)
30             r = r->next;
31         cout<<r->next->data<<" ";
32         r->next = r->next->next;
33         r = r->next;
34     }
35     return 0;
36 }
```

■ 判断题

28. node 这个结构体在内存空间中占 8 字节。 ()
29. 程序中有 new 却没有做 delete, 输入 n 大于 10 必然会导致程序运行时崩溃。 ()
30. 若输入为 8 5, 去掉第 15 行, 程序的运行结果不会改变。 ()
31. 本程序用到了双向链表的技巧。 ()

■ 选择题

32. 若输入为 8 5, 则输出是 ()。
- A. 5 2 7 8 1 4 6 3 B. 5 2 8 7 1 4 6 3
C. 5 2 8 7 4 1 6 3 D. 5 2 8 7 1 4 3 6
33. 若输入为 11 4, 则输出是 ()。
- A. 4 8 1 6 11 7 3 2 5 10 9 B. 4 8 6 1 11 7 3 2 5 10 9
C. 4 8 1 11 6 7 3 2 5 10 9 D. 4 8 1 6 11 7 2 3 5 10 9
34. (4 分) 程序运行的时间复杂度是 ()。
- A. $O(n^2)$ B. $O(\log n)$ C. $O(nm)$ D. $O(m)$

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 交叉字符串问题。给出 3 个字符串 s_1 、 s_2 和 s_3 , 判断 s_3 是否可以由 s_1 和 s_2 两个字符串经过交叉组合而成, 组合过程不能改变字符在 s_1 和 s_2 中的原本顺序。如 $s_1 = \text{"aabcc"}$, $s_2 = \text{"dbbca"}$, $s_3 = \text{"aadbcbcbac"}$ 或 $s_3 = \text{"aadbbaaccc"}$ 。给了两种 s_3 的情况, 第一种情况下答案是 yes, 第二种情况下答案是 no, 因为找不到任何一种 s_1 和 s_2 的交叉组合方式可以组合成 s_3 。

输入格式:

第 1 行输入一个整数 N 。第 2 行到第 $N+1$ 行输入 3 个字符串对 s_1, s_2, s_3 , s_1 和 s_2 的长度都不超过 200, s_3 的长度等于 s_1 和 s_2 的长度之和。

输出格式:

每个测试样例输出一行, yes 代表可以构成, no 代表不能。

输入样例:

```
3
cat tree tcraete
cat tree catrtee
cat tree cttaree
```

输出样例:

```
yes
yes
No
```

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 string str1,str2,str;
04 bool pos;
05 int vis[205][205];
06 void dfs(int s1,int s2,int s){
07     if(s1==str1.length() && s2==str2.length()){
08         ①;
09         return;
10     }
11     if( ② )
12         return;
13     if( ③ )
14         return;
15     vis[s1][s2]=1;
16     if(str1[s1]==str[s])
17         ④;
18     if(str2[s2]==str[s])
19         dfs(s1,s2+1,s+1);
20 }
21 int main()
22 {
23     int t,n;
24     cin>>t;
25     while(t--){
26         cin>>str1>>str2>>str;
27         ⑤;
28         memset(vis,0,sizeof(vis));
29         dfs(0,0,0);
30         if(pos)
31             cout<<"yes"<<endl;
32         else
33             cout<<"no"<<endl;
34     }
35     return 0;
36 }
```

35. ①处应填 ()。

A. pos=true

B. pos=false

C. vis[s1][s2]=1

D. vis[s1][s2]=0

36. ②处应填 ()。

- A. `str1[s1]==str[s] && str2[s2]!=str[s]`
- B. `str1[s1]!=str[s] && str2[s2]!=str[s]`
- C. `str1[s1]!=str[s] && str2[s2]==str[s]`
- D. `str1[s1]==str[s] && str2[s2]==str[s]`

37. ③处应填 ()。

- A. `vis[s1][s2]`
- B. `vis[s1][s2]==0`
- C. `pos==true`
- D. `pos==false`

38. ④处应填 ()。

- A. `dfs(s1,s2,s+1)`
- B. `dfs(s1+1,s2+1,s)`
- C. `dfs(s1+1,s2,s+1)`
- D. `dfs(s1+1,s2+1,s+1)`

39. ⑤处应填 ()。

- A. `vis[s1][s2]==1`
- B. `vis[s1][s2]==0`
- C. `pos==true`
- D. `pos==false`

(2) 设有 N 堆沙子排成一排, 编号为 $1, 2, \dots, N$ (N 小于 1000), 每堆沙子有一定的数量, 用 $a[k]$ 表示第 K 堆沙子的数量值。现在要将 N 堆沙子归并成为一堆, 归并的过程为每次只能将相邻的两堆沙子堆成一堆, 合并后的这堆沙子的代价为这两堆沙子的数值和, 这样经过 $N-1$ 次归并之后, 最后成为一堆。不同的归并方案的总代价值是不同的。现给出 N 堆沙子的数量后, 请找出一种合理的归并方法, 使总的归并代价最小。输入格式:

输入有 $n+1$ 行, 第 1 行为 n 的值, 从第 2 行开始, 为 n 个正整数, 表示各堆沙子的数量值。

输出格式:

只有一行, 表示最小的总代价, 结果保证小于 2^{31} 。

输入样例:

10
12
3
13
7

8
23
14
6
9
34

输出样例:

398

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05     int a[105],s[105],f[100][100];
06     int j,n,p,m;
07     cin>>n;
08     s[0]=0;
09     for(int i=1;i<=n;i++)
10     {
11         cin>>a[i];
12         ①;
13         f[i][i]=0;
14     }
15     for(p=1;p<=n-1;p++)
16         for(int i=1;i<n;i++)
17         {
18             ②;
19             if(j<=n)
20             {
21                 m=INT_MAX;
22                 for(int k=i;k<j;k++)
23                     m=min(m, ③);
24                 ④;
25             }
26         }
27     cout<<⑤;
28     return 0;
29 }
```

40. ①处应填 ()。

A. $s[i]=s[i-1]+a[i-1]$

B. $s[i]=s[i-1]+a[i]$

C. $s[i]=s[i-1]+a[i+1]$

D. $s[i]=\min(s[i-1],a[i])$

41. ②处应填 ()。

A. $j=i+p$

B. $j=i+p-1$

C. $j=i+p+1$

D. $j=\max(i,p)$

42. ③处应填 ()。

A. $f[i][k] + f[k+1][j]$

B. $f[i][k] + f[k][j]$

C. $f[i][k-1] + f[k+1][j]$

D. $f[i][k+1] + f[k+1][j]$

43. ④处应填 ()。

A. $f[i][j] = m + s[j] - s[i]$

B. $f[i][j] = m + s[j] - s[i-1]$

C. $f[i][j] = m + s[j-1] - s[i]$

D. $f[i][j] = m + s[j-1] - s[i-1]$

44. ⑤处应填 ()。

A. $f[0][n]$

B. $f[0][n-1]$

C. $f[1][n]$

D. $f[1][n-1]$



扫码获取
答案解析

提高组 CSP-S 2024 初赛模拟卷 1

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 表达式 $(8 \% (-6))$ 与 $(-8 \% 6)$ 的值为 ()。
A. -2, -2 B. 2, -2 C. -2, 2 D. 2, 2
- 设根结点深度为 0，一棵深度为 h 的满三叉树，即除最后一层无任何子结点外，每一层上的所有结点都有 3 个子结点的树，共有 () 个结点。
A. $(3^{h+1}-1)/2$ B. 3^{h-1} C. 3^h D. $(3^{h+1}-1)/2$
- 以下不属于调试工具 GDB 中的命令的是 ()。
A. printf B. display C. next D. step
- 以下哪个不属于算法的最本质特征？ ()
A. 有穷性 B. 确定性 C. 先进性 D. 确切性
- 以下哪个不属于哈希冲突的常见处理方法？ ()
A. 哈希法 B. 线性探查法 C. 二次探查法 D. 固定分配法
- 单源最短路问题的常用算法不包含 ()。
A. Bellman-Ford B. Dijkstra C. SPFA D. Kruskal
- 设某算法的时间复杂度函数的递推方程是 $F(n) = F(n-1) + 2n$ (n 为正整数) 及 $F(0) = 0$ ，则该算法的时间复杂度为 ()。
A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log n)$
- 一棵二叉树一共有 19 个结点，其叶子结点不可能有 () 个。
A. 11 B. 10 C. 9 D. 1
- 关于拓扑排序，下面的说法哪个是正确的？ ()
A. 拓扑排序只能用广度优先遍历实现